

STATISTICAL STRUCTURED PREDICTION

Joan Andreu Sánchez
jandreu@prhlt.upv.es



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



VIRTUAL TEACHING

Purpose:

Provision of the higher education public service
(art. 1 LOU)

Responsible:

Universitat Politècnica de València.

**Rights of access, rectification, deletion,
portability, limitation or opposition to the
treatment in accordance with privacy
policies:**

<http://www.upv.es/contenidos/DPD/>

Intellectual property:

Exclusive use in the virtual classroom
environment.

Dissemination, distribution or disclosure of
recorded classes, particularly on social networks
or services dedicated to sharing notes, is
prohibited.

Violation of this prohibition may generate
disciplinary, administrative or civil liability.



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



DOCENCIA VIRTUAL

Finalidad:

Prestación del servicio Público de educación
superior (art. 1 LOU)

Responsable:

Universitat Politècnica de València.

**Derechos de acceso, rectificación, supresión,
portabilidad, limitación u oposición al
tratamiento conforme a políticas de
privacidad:**

<http://www.upv.es/contenidos/DPD/>

Propiedad intelectual:

Uso exclusivo en el entorno de aula virtual.

Queda prohibida la difusión, distribución o
divulgación de la grabación de las clases y
particularmente su compartición en redes sociales
o servicios dedicados a compartir apuntes.

La infracción de esta prohibición puede generar
responsabilidad disciplinaria, administrativa o civil



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Técnicas Fundamentales	R.F. y Aprendizaje Computacional Lingüística Computacional
Reconocimiento de Formas	Predicción Estructurada Estadística Redes Neuronales Artificiales Aplicaciones de Reconocimiento de Formas
Tecnologías del Lenguaje	Traducción Automática Reconocimiento Automático del Habla Aplicaciones de la Lingüística Computacional
Técnicas Complementarias	Aprendizaje Automático Avanzado Reconocimiento de Escritura Biometría Visión por Computador

Focus of Course

- Give students an overview of the different topics that allow them to understand the basic concepts:
 - What is structured prediction?
 - What problems will we try to address?
 - How to make predictions and learn the models in structured output spaces?
 - What are the different computational challenges for structured prediction?
- We will deal with the rigorous design of algorithms and make intensive use of mathematics, but nothing too hard.



“There is nothing more practical than a good theory”.

Kurt Lewin

Syllabus

1. Introduction
2. Models for Statistical Structured Prediction
3. Making Prediction: Decoding and Inference
4. Model Parameter Estimation

Pattern Recognition and Machine Learning

- Christopher M. Bishop: *Pattern Recognition and Machine Learning*. Springer.
- Kevin P. Murphy: *Machine Learning: A Probabilistic Perspective*. The MIT Press.
- Daphne Koller and Nir Friedman: *Probabilistic Graphical Models: Principles and Techniques*. MIT Press.
- Richard O. Duda, Peter E. Hart and David G. Stork: *Pattern Classification* (2nd ed.). Wiley Interscience.

Statistical Models for Natural Language Processing

- Christopher D. Manning and Hinrich Schütze: *Foundations of Statistical Natural Language Processing*. The MIT Press.
- Noah A. Smith: *Linguistic Structure Prediction*. Morgan & Claypool.
- Daniel Jurafsky and James H. Martin: *Speech and Language Processing*. Prentice Hall (2nd ed).

Lectures

Monday 18:00 - 21:00 (10 sessions of 3 hours)

J.A. Sánchez November 2, first session

Assessment

- 2 Collections of selected exercises (theoretical and practical)
 - 1 Multiple-choice test 06-02-2026
 - 1 Recovery exam 12-02-2026
- [Multiple-choice test and theoretical/practical exercises]

	delivery	deadline
Q1	01 - 12 - 2025	22 - 12 - 2025
Q2	26 - 01 - 2026	02 - 02 - 2026

Tutoring

Tutoring available by previous appointment

Joan Andreu Sánchez <jandreu@prhlt.upv.es> (office: 1D15)

1. Introduction

1.1. Structured Prediction

1.2. Predicting Sequences

- Weighted Finite-State Transducers and Automata

1.3. Syntactic Parsing

- Context-Free Grammars



- Input observation; $x \in \mathcal{X}$ can be any kind of object.
- Output hypothesis; $y \in \mathcal{Y}$ is a real number: $\mathcal{Y} = \{1, \dots, K\}$ or $y \in \mathbb{R}$.
- (Non-structured) prediction function; $f : \mathcal{X} \rightarrow \mathbb{R}$ assigns a hypothesis $y = f(x)$ to each entry x .
 - Binary classification: $y \in \{-1, 1\}$.
 - Multiclass classification: $y \in \{1, \dots, K\}$.
 - Regression: $y \in \mathbb{R}$.

Algorithms for predicting non-structured output data

Naive Bayes classifier

Regression

Logistic Regression

Fisher's linear discriminant

Perceptron algorithm

K-Nearest Neighbor

Support Vector Machines

Random Forests

Neural Networks

...

What if the space of outputs is much larger and more structured?

Say trees, or in general, graphs.

Machine Translation

Input: Text sequences

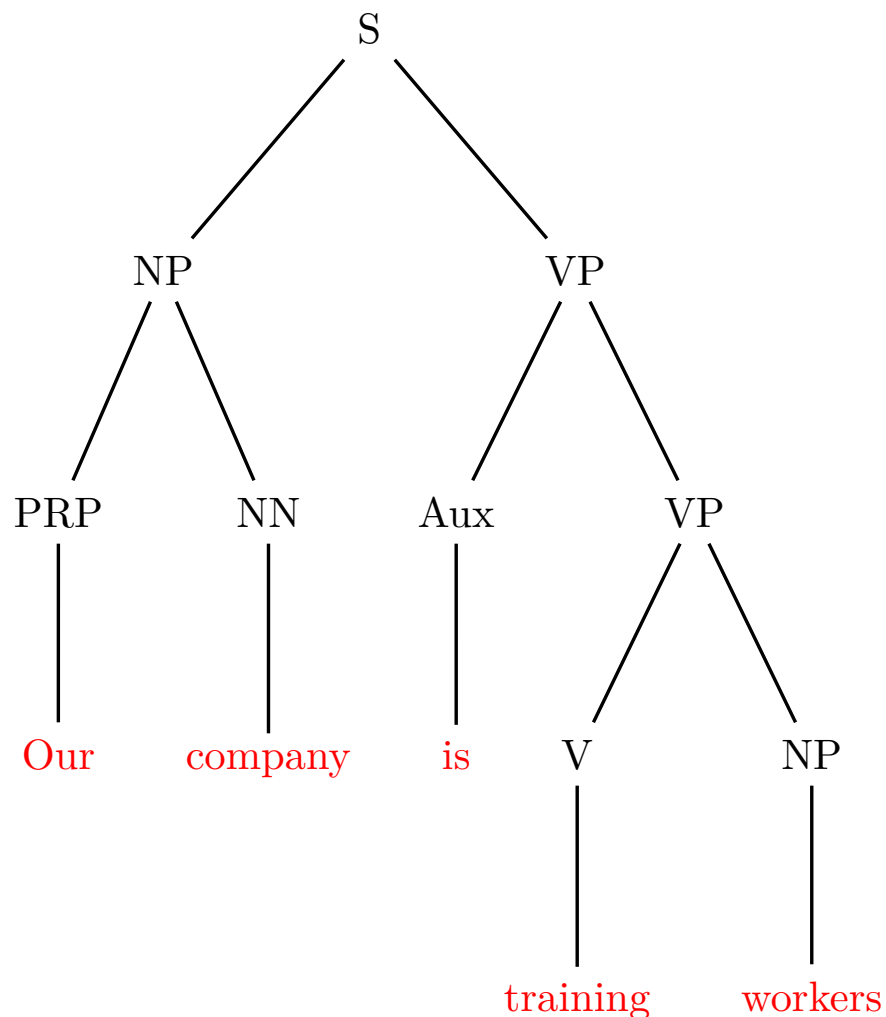
Output: Text sequences

The screenshot displays a web-based machine translation interface. At the top, the word "Translate" is written in red, and "Translation" is written in blue with a shadow effect. Below this, there are two sets of language selection buttons. The left set includes "English", "Spanish", "French", and "Latvian - detected" (which is highlighted with a blue arrow pointing to it). The right set includes "English", "Spanish", and "Arabic". A blue arrow points from the "Latvian - detected" button to the "English" button in the right set. A blue button labeled "Translate" is positioned to the right of the language buttons. Below the language selection, there are two text boxes. The left box contains the original Latvian text: "Bulgārija (bulgāru: България), oficiāli Bulgārijas Republika (Република България) ir valsts Austrumeiropā, kas atrodas Balkānos. Tā robežojas ar piecām citām valstīm: ziemeļos ar Rumāniju, rietumos ar Serbiju un Maķedoniju, dienvidos ar Grieķiju". A blue arrow points from the text "Original selected text fragment" (written in blue with a shadow) to this box. The right box contains the translated English text: "Bulgaria (Bulgarian: България), officially the Republic of Bulgaria (Република България) is a country in Eastern Europe, in the Balkans. It borders five other countries: Romania to the north, to the west by Serbia and Macedonia, to the south by Greece and Turkey. East it is washed by the Black Sea. Bulgaria has an area of 110,993 km² and has a population of about 7.35 million inhabitants. The capital is Sofia. Sofia is also the largest city in the country. The next largest cities are Plovdiv and Varna." A blue arrow points from the "Translate" button to this box. Both text boxes have a close button (X) and a speaker icon (audio playback) in the top right corner. The right box also has a star icon, a list icon, and a pencil icon in the bottom left corner, and a speaker icon and a checkmark icon in the bottom right corner.

Syntactic Parsing

Input: Text sequences

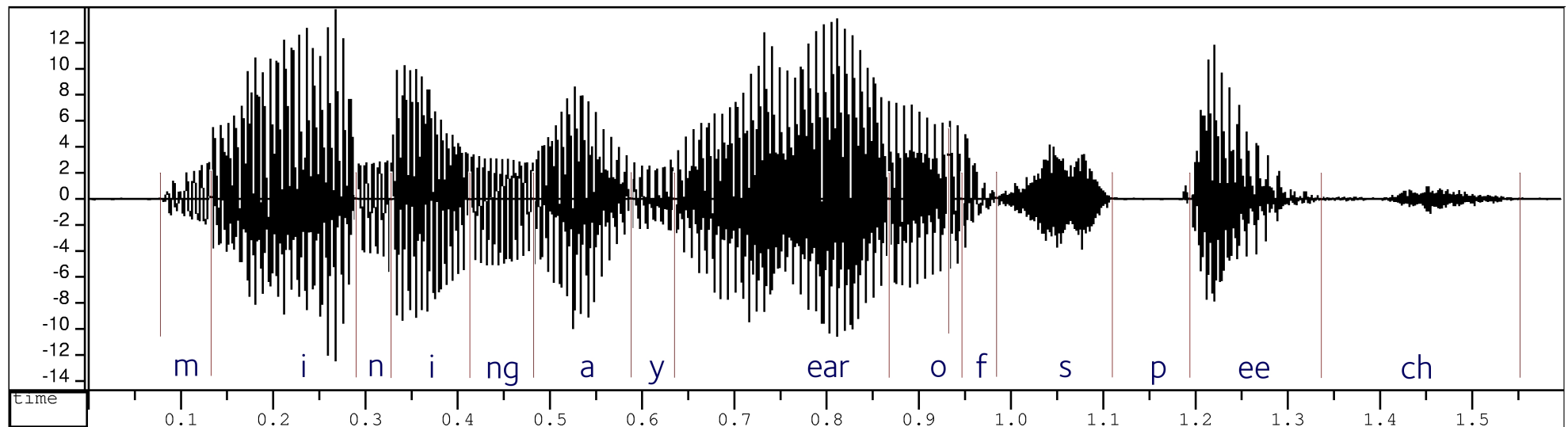
Output: Parse trees



Automatic Speech Recognition

Input: Speech signals

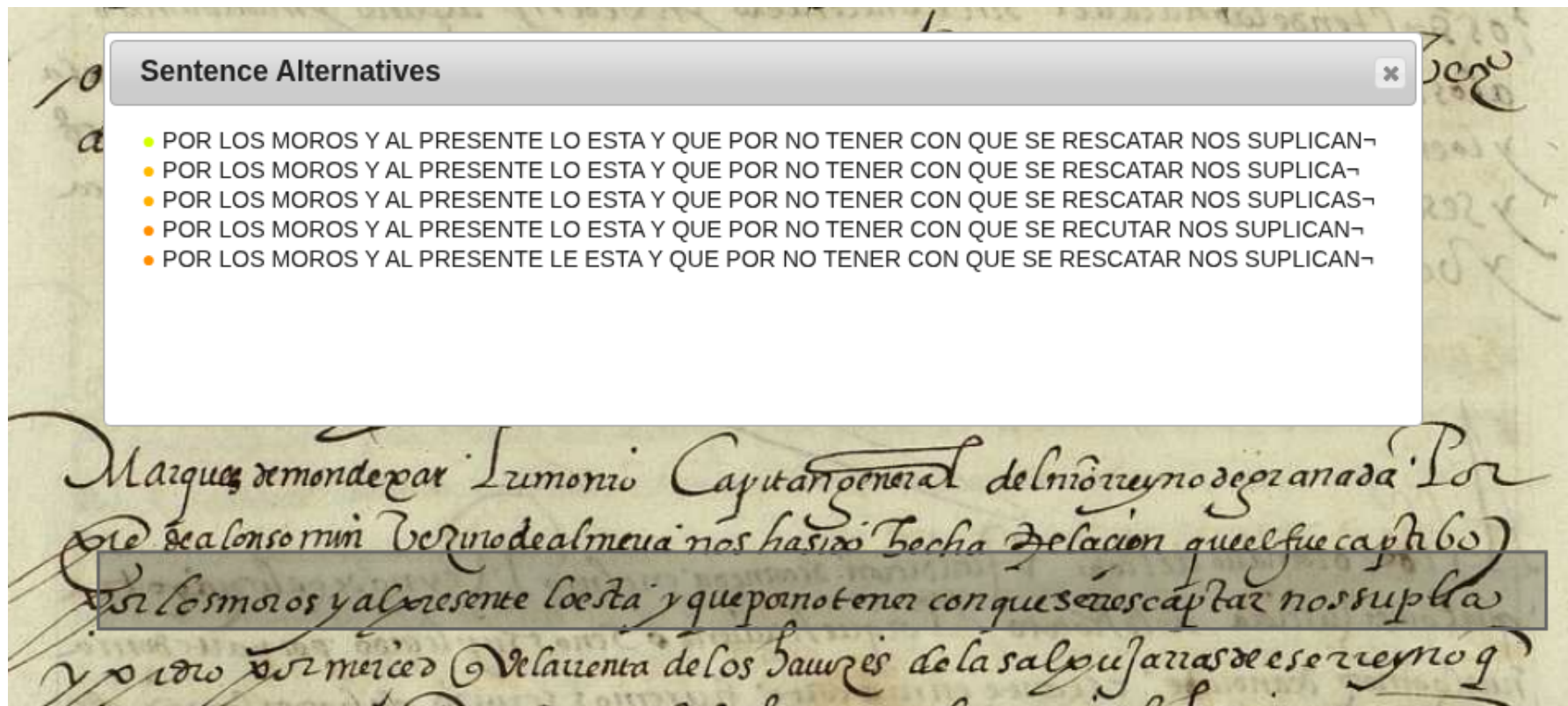
Output: Transcribed text sequences



Handwriting Text Recognition

Input: Images

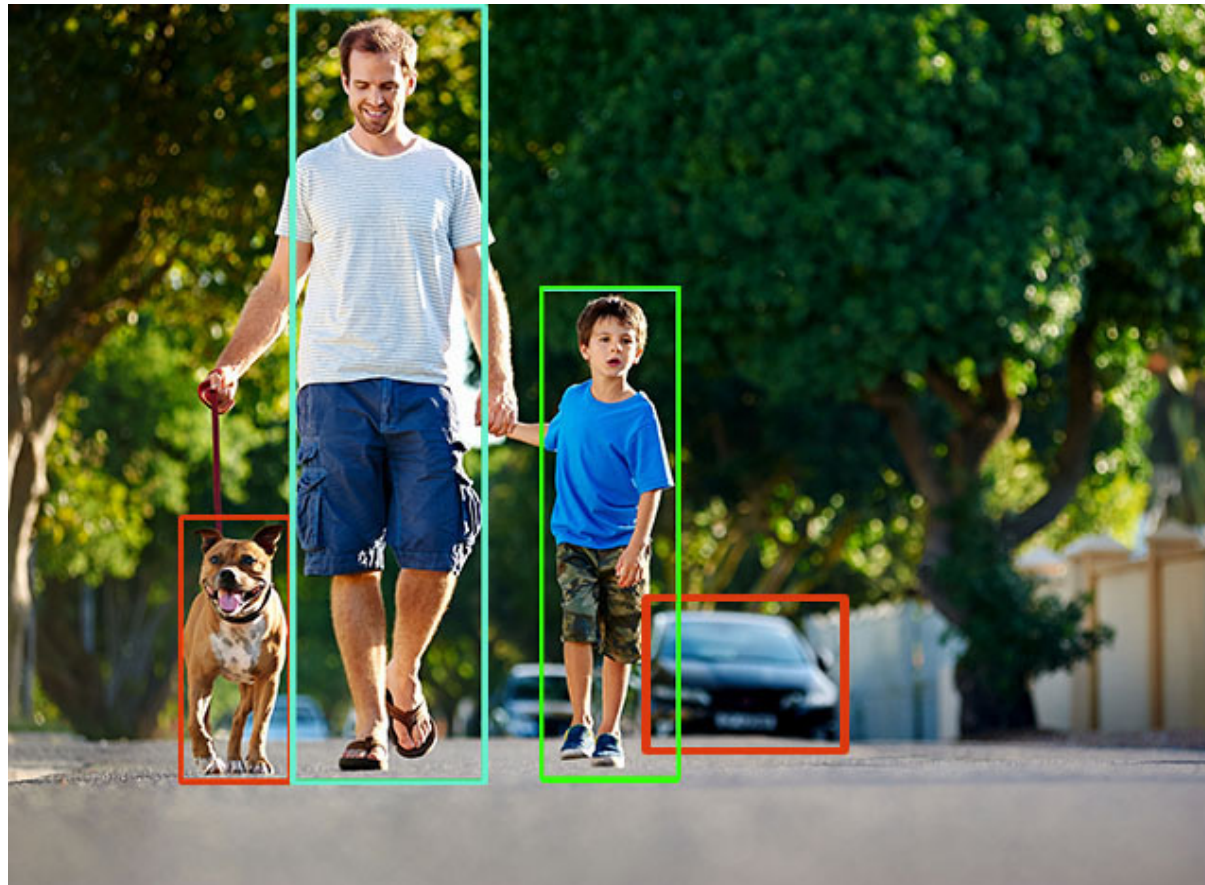
Output: Transcribed text sequences



Scene Analysis

Input: Images

Output: Scene layout graphs

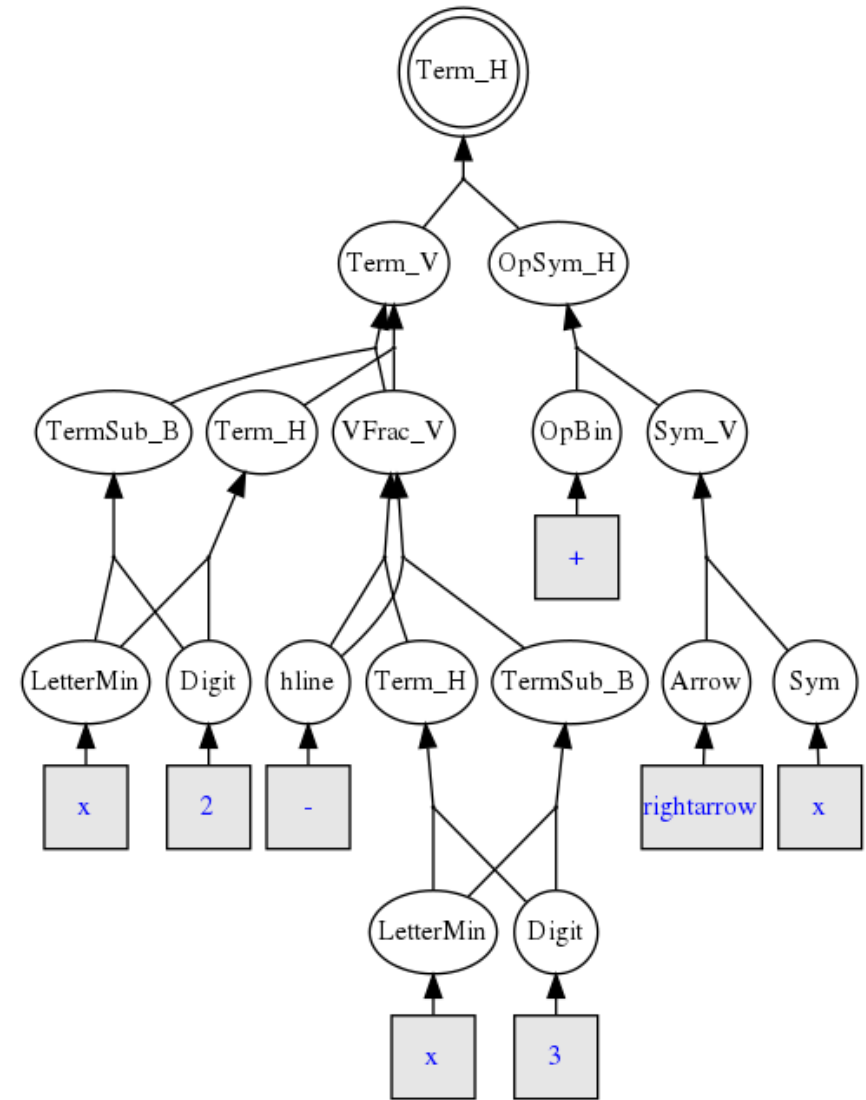


Mathematical Expression Recognition

Input: Images

Output: Hypergrphs

$$\frac{x_2}{x_3} + \vec{x}.$$



⇒ Mathematical Expression Recognition

Outputs

➤ Natural Language Processing:

- Part-of-Speech tagging (sentences)
- Parsing (parse trees)
- Machine Translation (sentences or word graphs)
- Information Extraction (sentences)

➤ Image Processing:

- Visual Scene Analysis (sentences or relationship graphs)
- Handwritten Text Recognition (sentences or word graphs)

➤ Speech Processing:

- Automatic transcription (sentences or word graphs)
- Text-to-Speech (audio signal)

➤ Bioinformatics:

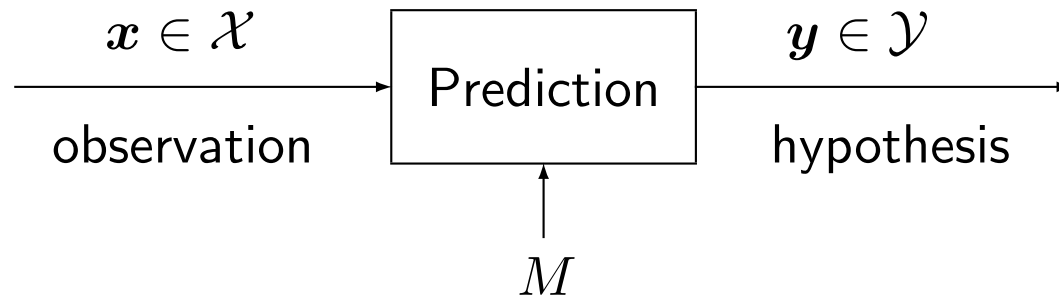
- Protein Structure Prediction (trees or graphs)

➤ Robotics:

- Planning (sequence of actions)



- Input observation; $x \in \mathcal{X}$ can be any kind of objects
- Output hypothesis; $y \in \mathcal{Y}$ is a complex structured object.
- (Structured) prediction function; $f : \mathcal{X} \rightarrow \mathcal{Y}$ which assigns a hypothesis $y = f(x)$ to each entry x .
 - y can be a sequence,
 - y can be a parse tree,
 - y can be a graph.



- Typically, $\mathcal{Y}$ is potentially infinite $\implies f(x) = y \in \mathcal{Y}$ will often be intractable.
- **(Structural) Solution:** We assume a specific structure in $\mathcal{Y}$, and we define a (finite) model M that allows us to characterize this structure: $Y(M) \subseteq \mathcal{Y}$

$$f(x) = y \in Y(M)$$

Depending on M , we can find polynomial solutions.

Sequences

- Text is a sequence of words or even letters,
- A spoken utterance is a sequence of parameter vectors,
- A video is a sequence of frames, ...

Models and motivation

Finite-State Acceptors: Compact representations of regular sets that are efficient to search, e.g. pattern matching, tokenization, compression.

Finite-State Transducers: Compact representations of rational binary relations that are efficient to search and combine, e.g. dictionaries, context-dependent rules.

Weighted Automata: Weights typically encode uncertainty (probabilities), e.g. n-gram language models, Hidden Markov Models.

WFSA and WFST-based operations are underpinned by algebraic structures called **semi-rings**.

Definition. A **semiring** is an algebraic system $(\mathbb{K}, \oplus, \otimes, \bar{0}, \bar{1})$ such that,

- $(\mathbb{K}, \oplus, \bar{0})$ is a **commutative monoid**^a with $\bar{0}$ as the identity element for $\oplus$,
- $(\mathbb{K}, \otimes, \bar{1})$ is a **monoid** with $\bar{1}$ as the identity element for $\otimes$,
- $\otimes$ distributes over $\oplus$: for all $a, b, c \in \mathbb{K}$,

$$(a \oplus b) \otimes c = (a \otimes c) \oplus (b \otimes c)$$

$$c \otimes (a \oplus b) = (c \otimes a) \oplus (c \otimes b)$$

- $\bar{0}$ is an annihilator for $\otimes$: for all $a \in \mathbb{K}$,

$$a \otimes \bar{0} = \bar{0} \otimes a = \bar{0}$$

This has implications for optimization, search, and combination algorithms such as determinization, shortest-path, and composition.

^a A monoid is an algebraic structure that supports a single associative binary operation and an identity element.

- **Product** $\otimes$: to compute the weight of a path
(product of the weights of constituent transitions).
- **Sum** $\oplus$: to compute the weight of a sequence
(sum of the weights of the paths labeled with that sequence).

Semiring	Set	$\oplus$	$\otimes$	$\bar{0}$	$\bar{1}$
Boolean	$\{ 0, 1 \}$	$\vee$	$\wedge$	0	1
Probability	$\mathbb{R}_+ \cup \{ +\infty \}$	$+$	$\times$	0	1
Log	$\mathbb{R} \cup \{ -\infty, +\infty \}$	$\oplus_{\log}$	$+$	$+\infty$	0
Tropical	$\mathbb{R}_+ \cup \{ +\infty \}$	$\min$	$+$	$+\infty$	0

- The **log semiring** is isomorphic to the **probability semiring** and
 $\oplus_{\log}$ is defined by: $x \oplus_{\log} y = -\log(e^{-x} + e^{-y})$
- The **tropical semiring** is derived from the **log semiring** using the
Viterbi approximation

[M. Mohri. *Semiring frameworks and algorithms for shortest-distance problems*. 2002], for more details.

Tropical Semiring example

Definitions

$$a \oplus b \stackrel{\text{def}}{=} \min(a, b)$$

$$a \otimes b \stackrel{\text{def}}{=} a + b$$

$$\bar{0} \stackrel{\text{def}}{=} +\infty$$

$$\bar{1} \stackrel{\text{def}}{=} 0$$

Examples

$$7 \oplus 4 = 4$$

$$1 \otimes 7 = 8$$

$$7 \oplus \bar{0} = 7$$

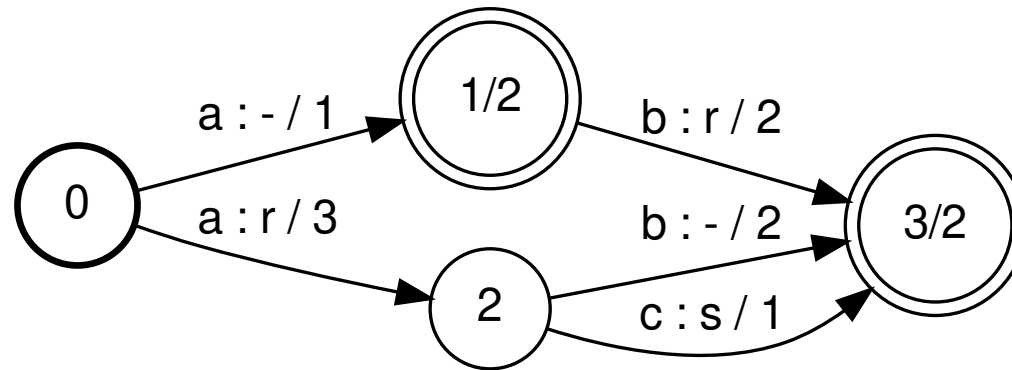
$$3 \otimes \bar{1} = 3$$

$$(4 \otimes 3) \oplus (2 \otimes 3) = 5$$

$$4 \oplus \bar{1} = 0$$

Definition. A **weighted transducer** T over a semiring $(\mathbb{K}, \oplus, \otimes, \bar{0}, \bar{1})$ is a tuple $T = (\Sigma, \Delta, Q, I, F, \delta, \lambda, \rho)$, where

- Finite input alphabet Σ and finite output alphabet Δ .
- States Q , initial states $I \subseteq Q$, and final states $F \subseteq Q$.
- Transition function $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times (\Delta \cup \{\epsilon\}) \times \mathbb{K} \times Q$.
- Initial weight function $\lambda : I \rightarrow \mathbb{K}$ and final weight function $\rho : F \rightarrow \mathbb{K}$.



Probability semiring $(\mathbb{K}, +, \times, 0, 1)$

$$[[T]](ab, r) = 16$$

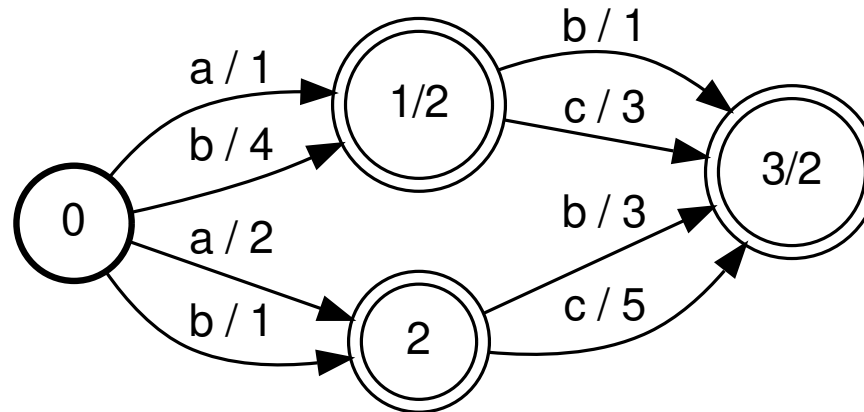
Tropical semiring $(\mathbb{K}, \min, +, \infty, 0)$

$$[[T]](ab, r) = 5$$

[Obtained from the OpenFst tool documentation]

Definition. A **weighted automaton** A over a semiring $(\mathbb{K}, \oplus, \otimes, \bar{0}, \bar{1})$ is a tuple $A = (\Sigma, Q, I, F, \delta, \lambda, \rho)$, where

- Finite input alphabet Σ .
- States Q , initial states $I \subseteq Q$, and final states $F \subseteq Q$.
- Transition function $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \mathbb{K} \times Q$.
- Initial weight function $\lambda : I \rightarrow \mathbb{K}$ and final weight function $\rho : F \rightarrow \mathbb{K}$.



Probability semiring $(\mathbb{K}, +, \times, 0, 1)$

$$[[T]](ab) = 14$$

Tropical semiring $(\mathbb{K}, \min, +, \infty, 0)$

$$[[T]](ab) = 4$$

[Obtained from the OpenFst tool documentation]

Path π

- $p[\pi]$ Path origin state
- $n[\pi]$ Path destination state
- $w[\pi]$ Path weight

Sets of paths

$P(I, x, F)$ Set of all paths from I to F with input label $x \in \Sigma^*$

$P(I, x, y, F)$ Set of all paths from I to F with input label $x \in \Sigma^*$ and output label $y \in \Delta^*$

Automata and transducers

Given an **automaton** $A = (\Sigma, Q, I, F, \delta, \lambda, \rho)$, for all $x \in \Sigma^*$

$$[[A]](x) = \bigoplus_{\pi \in P(I, x, F)} \lambda(p(\pi)) \otimes w(\pi) \otimes \rho(n(\pi))$$

Given a **transducer** $T = (\Sigma, \Delta, Q, I, F, \delta, \lambda, \rho)$, for all $x \in \Sigma^*$, $y \in \Delta^*$

$$[[T]](x, y) = \bigoplus_{\pi \in P(I, x, y, F)} \lambda(p(\pi)) \otimes w(\pi) \otimes \rho(n(\pi))$$

Rational operations

Sum (union)

$$[[T_1 \oplus T_2]](x, y) = [[T_1]](x, y) \oplus [[T_2]](x, y)$$

Product (Concat.)

$$[[T_1 \otimes T_2]](x, y) = \bigoplus_{\substack{x=x_1x_2 \\ y=y_1y_2}} [[T_1]](x_1, y_1) \otimes [[T_2]](x_2, y_2)$$

Closure

$$[[T^*]](x, y) = \bigoplus_{n=0}^{\infty} [[T^n]](x, y)$$

Unary operations

Reversal

$$[[T^R]](x, y) = [[T]](x^R, y^R)$$

Inversion

$$[[T^{-1}]](x, y) = [[T]](y, x)$$

Projection

$$[[\downarrow T]](x) = \bigoplus_y [[T]](x, y)$$

Binary Operations

Composition

$$[[T_1 \circ T_2]](x, y) = \bigoplus_z [[T_1]](x, z) \otimes [[T_2]](z, y)$$

Intersection

$$[[A_1 \cap A_2]](x) = [[A_1]](x) \oplus [[A_2]](x)$$

Difference

$$[[A_1 - A_2]](x) = [[A_1 \cap \overline{A_2}]](x)$$

Optimization algorithms

ϵ -Removal: Creates an equivalent ϵ -free transducer.

Determinization: Creates an equivalent deterministic transducer.

Pushing: Pushes arc weights forward or backward, accumulating and/or distributing them according to the semiring

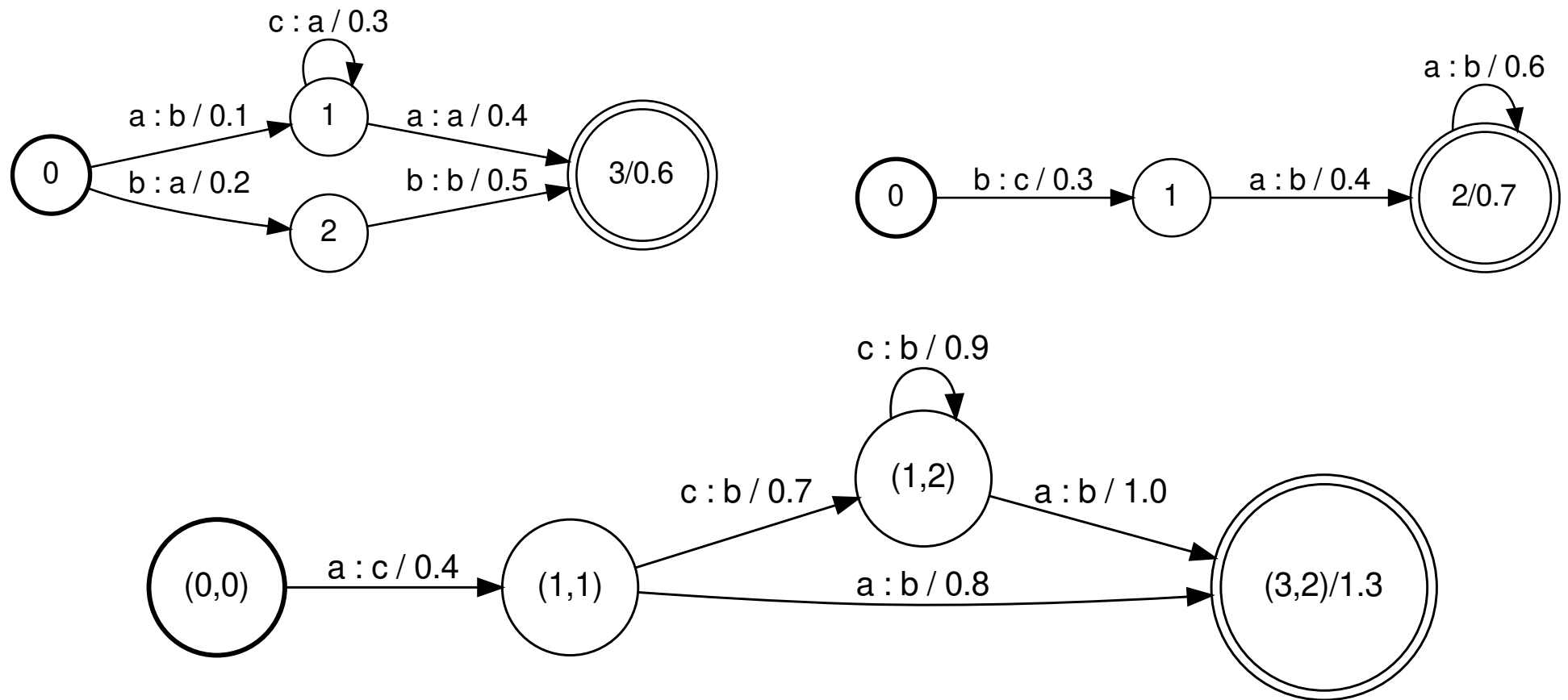
Minimization: Creates an equivalent minimal deterministic transducer.

Shortest-distance algorithms: **Shortest path**, and **N-Shortest paths**.

[M. Mohri. *Semiring frameworks and algorithms for shortest-distance problems*. 2002], for more details.

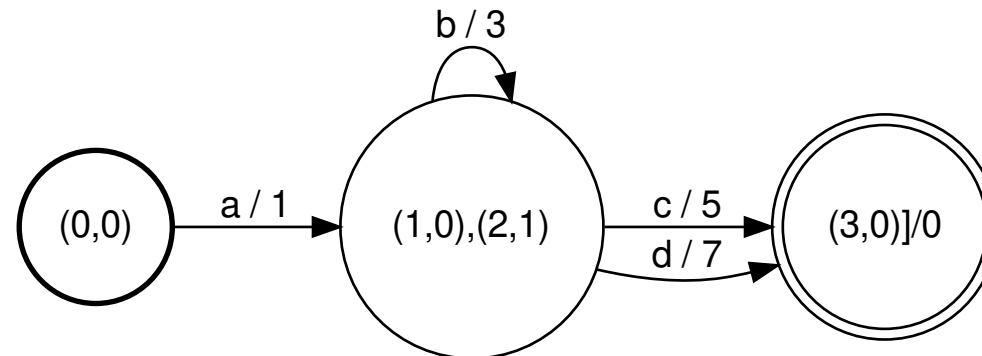
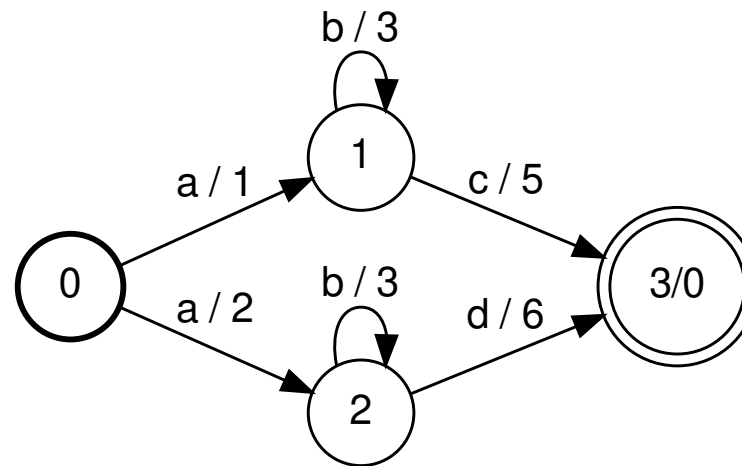
Definition:
$$[[T_i \circ T_2]](x, y) = \bigoplus_z [[T_1]](x, z) \otimes [[T_2]](z, y)$$

Example: Weighted automaton over the tropical semiring



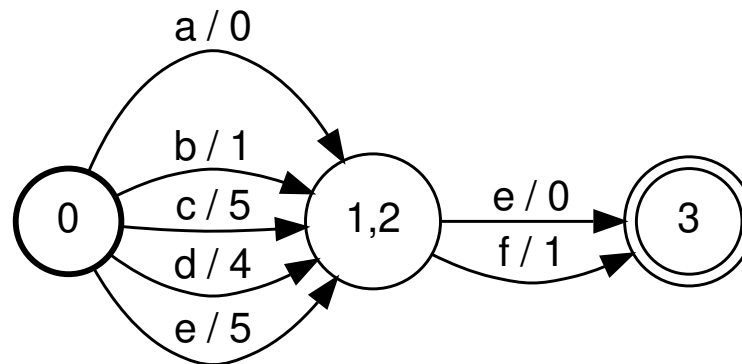
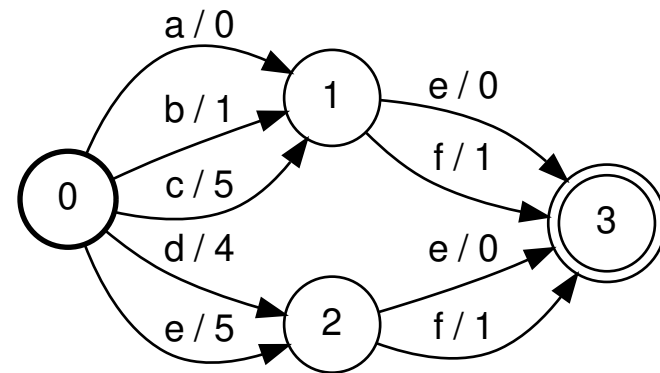
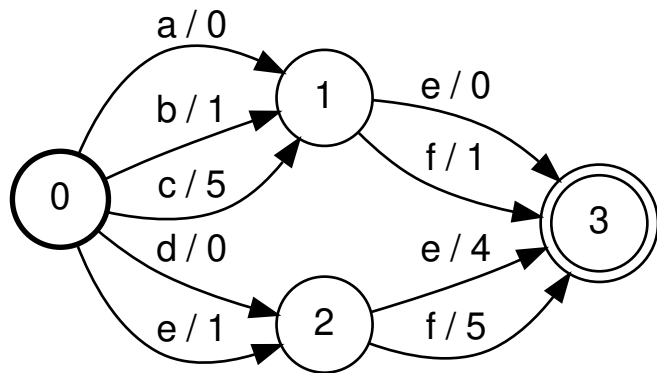
Definition: Creates an equivalent deterministic weighted automaton/transducer

Example: Weighted automaton over the tropical semiring



Definition: Computes a minimal equivalent deterministic machine while preserving the input language and and weight/path properties of the original

Example: Weighted automaton over the tropical semiring



General Background

- J.E.Hopcroft, J.D.Ullman: *Introduction to Automata Theory, Languages, and Computation*. Addison Wesley, 1979.
- T.H. Cormen, C.E.Leiserson, R.L.Rivest: *Introduction to Algorithms*. The MIT Press, 1992.

WFST and WFSA applications

- M.Mohri, F.Pereira, M.Riley: *Speech Recognition with Weighted Finite-State Transducers*. In *Springer Handbook of Speech Processing*. Springer, 2008.
- M.Mohri: *Weighted Automata Algorithms*. In *Handbook of Weighted Automata*. Monographs in Theoretical Computer Science. Springer, 2009.
- A.Argueta, D.Chiang: *Composing Finite State Transducers on GPUs*. Proc. of the ACL, p.pages 2697–2705. Melbourne, Australia, 2018.

Software

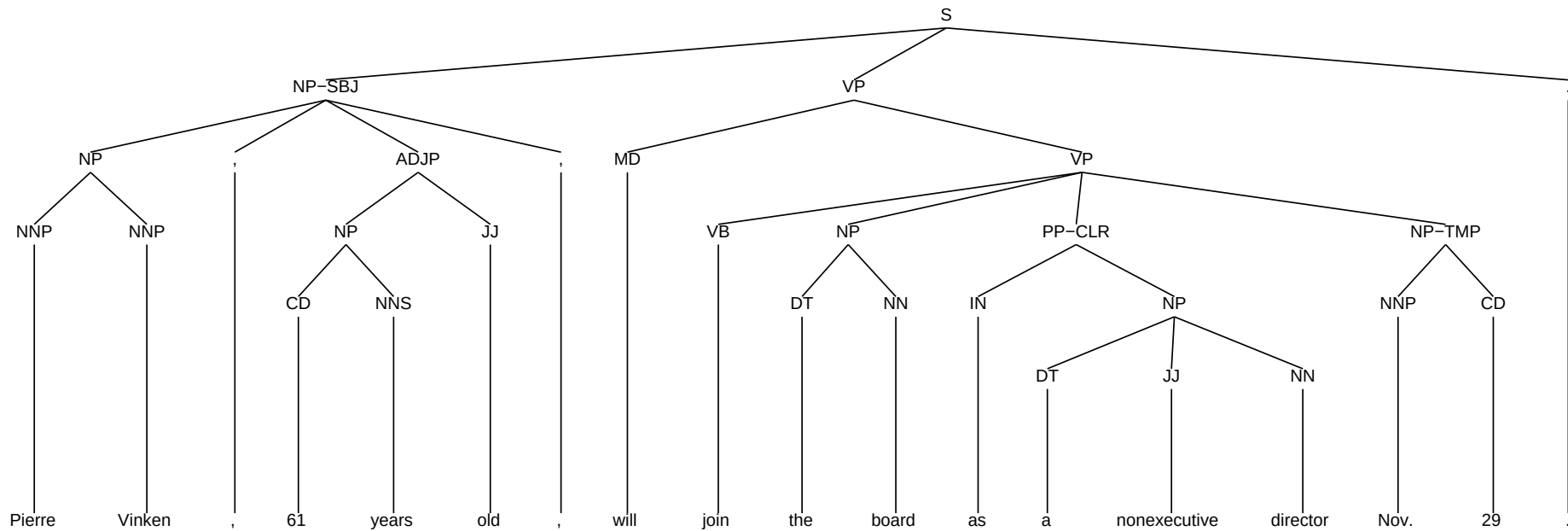
OpenFst is a open-source C++ library for weighted finite state transducers developed at Google.

More information available at <http://www.openfst.org>

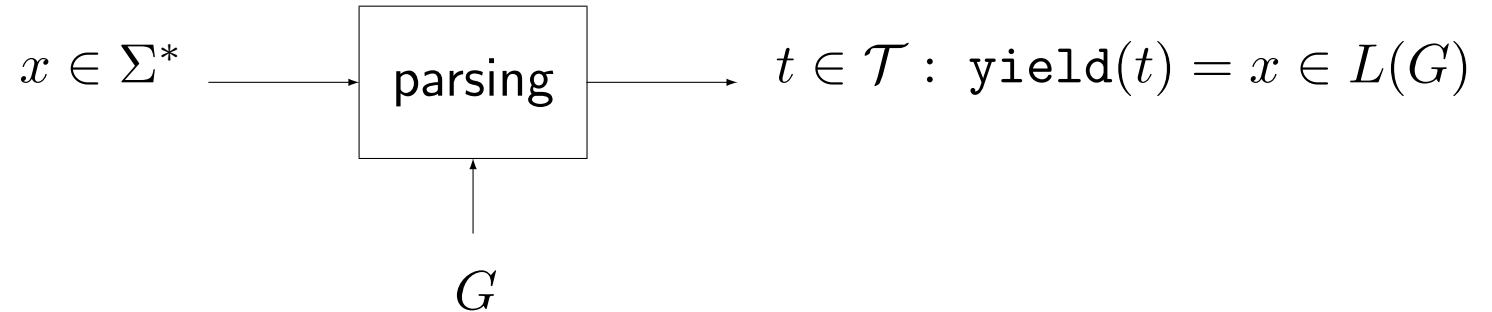
- C.Allauzen, M.Riley, J.Schalkwyk, W.Skut, M.Mohri: *OpenFst: A General and Efficient Weighted Finite-State Transducer Library*. Proc. of the CIAA. Prague, CZ, 2007.

Example: *“Pierre Vinken, 61 years old, will join the board as a nonexecutive director Nov. 29.”* [Marcus et al., 1993]

((S (NP-SBJ (NP (NNP Pierre) (NNP Vinken)) (,,) (ADJP (NP (CD 61) (NNS years)) (JJ old)) (,,)) (VP (MD will) (VP (VB join) (NP (DT the) (NN board)) (PP-CLR (IN as) (NP (DT a) (JJ nonexecutive) (NN director)))) (NP-TMP (NNP Nov.) (CD 29)))) (..)))



➤ Parsing as a search problem



➤ Grammatical models: reasons for use

- Grammatical models are the simplest and most natural model for tree structures
- Formal (mathematical) framework is well known
- Compact models (small number of free parameters)
- Good behavior against the problem of ambiguity
- Context-free grammatical models represent well long-term dependencies of syntactic and semantic constraints of Natural Language

➤ Context-Free Grammars $G = (\Sigma, N, S, \mathcal{P})$

Σ a set of **terminal symbols**

N a set of **non-terminals symbols** (or variables): $N \cap \Sigma = \emptyset$

S a distinguished **start symbol**: $S \in N$

$\mathcal{P}$ a set of **rules** (or productions): $(A \rightarrow \alpha) \in \mathcal{P}; A \in N; \alpha \in (N \cup \Sigma)^*$

➤ Direct derivation: $\delta A \gamma$ **directly derives** $\delta \alpha \gamma$ or

$$\delta A \gamma \Rightarrow \delta \alpha \gamma \quad \textbf{iff} \quad \exists (A \rightarrow \alpha) \in \mathcal{P}; \delta, \gamma \in (N \cup \Sigma)^*$$

➤ Derivation: α **derives** β or

$$\alpha \xRightarrow{*} \beta \quad \textbf{iff} \quad \exists \alpha_0, \dots, \alpha_m \in (N \cup \Sigma)^*: \alpha = \alpha_0 \Rightarrow \alpha_1 \Rightarrow \dots \alpha_{m-1} \Rightarrow \alpha_m = \beta$$

➤ Language generated by a grammar: $L(G) = \{x \mid x \in \Sigma^* : S \xRightarrow{+} x\}$

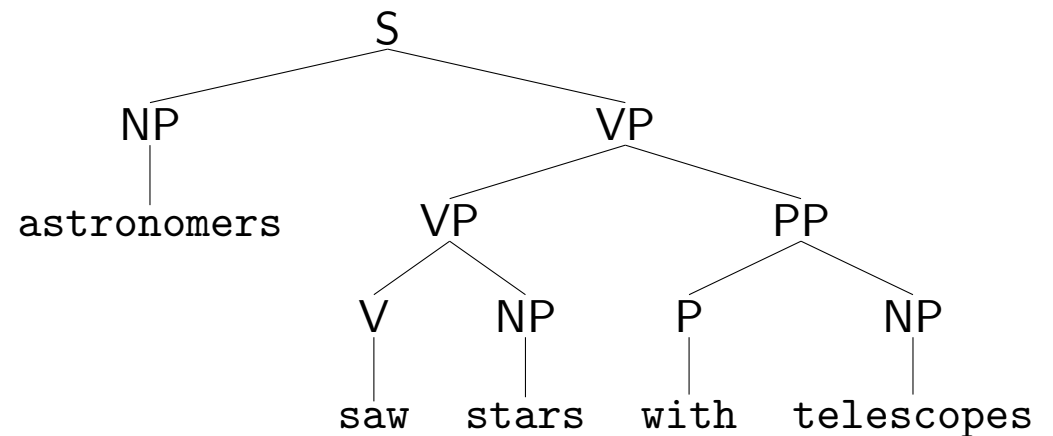
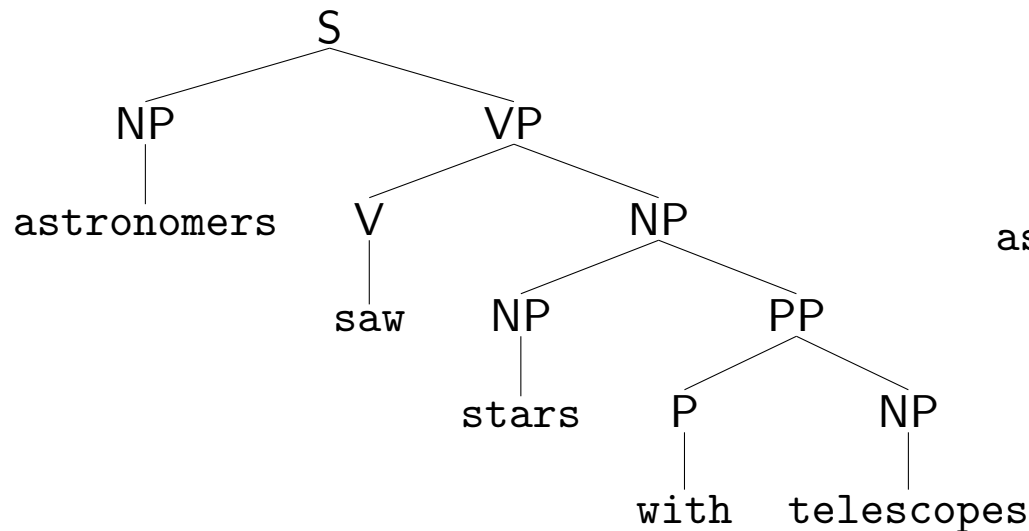
➤ Theorem $x \in L(G) \quad \textbf{iff} \quad S \xRightarrow{+} x \quad \textbf{iff} \quad \exists t \in \mathcal{T} : \text{yield}(t) = x$

Example: A simple Context-Free Grammars

[Manning and Schütze, 2002]

$S \rightarrow NP VP$	$VP \rightarrow V NP$	$V \rightarrow \text{say}$	$NP \rightarrow \text{say}$
$NP \rightarrow NP PP$	$VP \rightarrow VP PP$	$NP \rightarrow \text{astronomers}$	$NP \rightarrow \text{stars}$
$PP \rightarrow P NP$	$P \rightarrow \text{with}$	$NP \rightarrow \text{ears}$	$NP \rightarrow \text{telescopes}$

$S \Rightarrow NP VP \Rightarrow \text{astronomers VP} \Rightarrow \text{astronomers V NP} \Rightarrow \text{astronomers say NP} \Rightarrow$
 $\text{astronomers say NP PP} \Rightarrow \text{astronomers say stars PP} \Rightarrow \text{astronomers say stars P NP}$
 $\Rightarrow \text{astronomers say stars with NP} \Rightarrow \text{astronomers say stars with telescopes}$



Algorithm 1: Cocke-Kasami-Younger

Input: $G = (\Sigma, N, S, \mathcal{P})$ in FNC and

$\mathbf{x} = x_1 \dots x_T \in \Sigma^*$

Output: Parsing table $t[i, j]$ $(1 \leq i, j \leq T)$;

$A \in t[i, i+l]$ **iff** $A \xRightarrow{*} x_{i+1} \dots x_{i+l}$

for $i : 0 \dots T-1$ **do**

$t[i, i+1] = t[i, i+1] \cup \{A : (A \rightarrow b) \in P; b = x_{i+1}\}$

for $l : 2 \dots T$ **do**

for $i : 0 \dots T-l$ **do**

for $k : 1 \dots l-1$ **do**

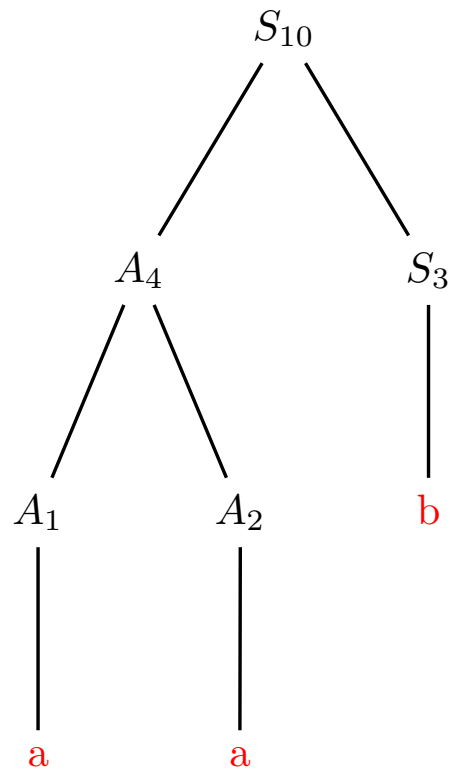
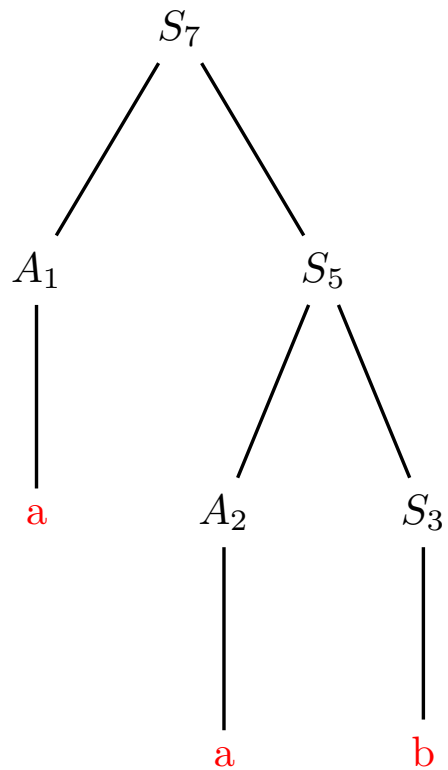
$t[i, i+l] = t[i, i+l] \cup \{A : (A \rightarrow BC) \in P ;$
 $B \in t[i, i+k]; C \in t[i+k, i+l]\} ;$

if $S \in t[0, T]$ **then** $x \in L(G)$ **else** $x \notin L(G)$;

CKY PARSING: EXAMPLE

$S \Rightarrow A S$
 $S \Rightarrow b$
 $A \Rightarrow A S$
 $A \Rightarrow A A$
 $A \Rightarrow a$

a a b



	$j = 1$	$j = 2$	$j = 3$
$i = 0$	1: (A, 0, 0)	4: (A, 1, 2)	7: (S, 1, 5) 8: (A, 1, 5) 9: (A, 1, 6) 10: (S, 4, 3) 11: (A, 4, 3)
$i = 1$		2: (A, 0, 0)	5: (S, 2, 3) 6: (A, 2, 3)
$i = 2$			3: (S, 0, 0)

$l = 3$ (from cell $i=0, j=3$ to $i=1, j=2$)
 $l = 2$ (from cell $i=1, j=2$ to $i=2, j=1$)
 $l = 1$ (from cell $i=2, j=1$ to $i=3, j=0$)

2. Models for Statistical Structured Prediction

2.1. Introduction: Statistical Models

2.2. Generative Models

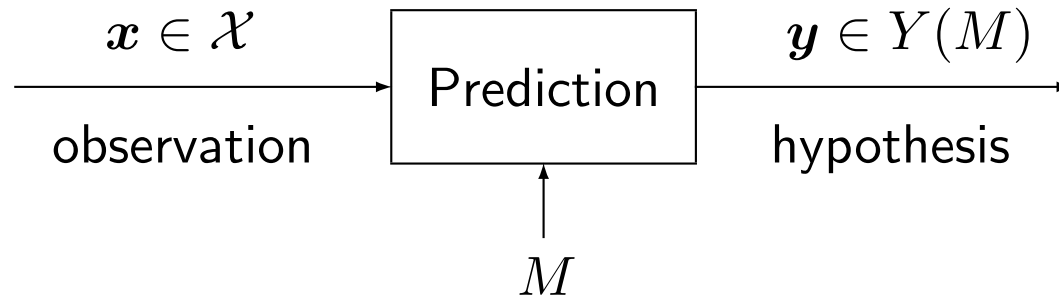
- Probabilistic Finite State Automata
- Probabilistic Context-Free Grammars

2.3. Probabilistic Graphical Models

- Bayesian Networks (Directed Graphical Models)
- Markov Random Fields (Undirected Graphical Models)

2.4. Discriminative Models

- Conditional Random Fields



➤ Real life

- **Fragility:** Lack of robustness due to an imperfect representation of input objects
[Uncertainty regarding representations of inputs]
- **Ambiguity:** Several interpretations (hypothesis) are plausible due to variability and difficulty of the task. [Uncertainty regarding outputs]
- **Incompleteness:** Inadequate models due to incomplete and insufficient knowledge
[uncertainty !]

➤ (Statistical) Solution Based on statistical decision theory $f : \mathcal{X} \rightarrow Y(M)$

$$\hat{\mathbf{y}} = f(\mathbf{x}) = \arg \max_{\mathbf{y} \in Y(M)} P(\mathbf{y} | \mathbf{x})$$

Discriminative models (conditional probability) $P(y \mid x)$

$$f(x) = \arg \max_{y \in Y(M)} P(y \mid x)$$

Generative models (join probability) $P(x, y)$

$$f(x) = \arg \max_{y \in Y(M)} P(x, y) = \arg \max_{y \in Y(M)} P(x \mid y) \cdot P(y)$$

Three Key Components

- **Statistical model:** used to replace the unknown real models

$$P(y|x) \text{ or } P(x|y) \text{ and } P(y) \quad \text{vs} \quad P_{\theta}(y|x) \text{ or } P_{\theta}(x|y) \text{ and } P_{\theta}(y)$$

Discriminant Function, Neural networks, Gaussian mixtures, models with hidden variables (HMM, alignment models), log-linear models, etc.

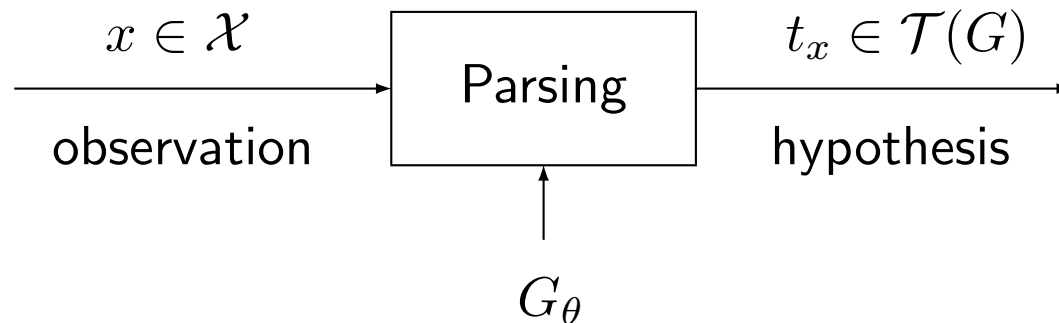
- **Decision rule (search or decoding):** Sometimes hard !

Dynamic Programming, A* search, etc.

- **Training criterion** to learn the unknown parameters θ from training data

Maximum Likelihood Estimation, Maximum a Posterior Estimation, Minimum Classification error, etc.

Specific algorithms depend on statistical models



$$t_x \in \mathcal{T}(G) : \text{yield}(t) = x \quad \text{iff} \quad S \overset{+}{\Rightarrow} x \quad \text{iff} \quad x \in L(G)$$

(Statistical) Solution based on statistical decision theory: **Probabilistic models**

$$x \in L(G) \quad \Longrightarrow \quad P(x; G_\theta) \quad \equiv \quad P(x; \theta) \quad \equiv \quad P_\theta(x)$$

where θ is the parameter vector of the probabilistic model G_θ

➤ Language $L \subseteq \Sigma^*$, given an alphabet Σ

➤ Language generated by a grammar $L(G) \subseteq \Sigma^*$,
given an alphabet Σ and a grammar $G = (\Sigma, N, S, \mathcal{P})$

$$L(G) = \{x \mid x \in \Sigma^* : S \xRightarrow{+} x\}$$

➤ Probabilistic language (L, ϕ) , given an alphabet Σ

➤ $L \subseteq \Sigma^*$ characteristic language

➤ $\phi : \Sigma^* \longrightarrow [0, 1]$ computable probabilistic function:

$$\text{i) } x \notin L \implies \phi(x) = 0 \quad \forall x \in \Sigma^*$$

$$\text{ii) } x \in L \implies 0 < \phi(x) \leq 1 \quad \forall x \in \Sigma^*$$

$$\text{iii) } \sum_{x \in L} \phi(x) = 1$$

➤ **Example** [Booth-Thompson,73]

Given the alphabet $\Sigma = \{a, b\}$, the following language is defined:

$$L = \{a^n b^n \mid n \geq 0\}, \quad \text{where } \phi(x) = 0, \quad \forall x \notin L \quad \text{and} \quad \phi(a^n b^n) = \frac{1}{e \, n!}$$

$$\sum_{x \in L} \phi(x) = \sum_{0 \leq n \leq \infty} \frac{1}{e \, n!} = \frac{1}{e} \sum_{0 \leq n \leq \infty} \frac{1}{n!} = \frac{1}{e} e = 1$$

➤ **Theorem** [Wetherell,80]

Let (L, ϕ) be an infinite probabilistic language, then for each $\epsilon > 0$ there exists an $n \geq 0$, such as:

$$|x| \geq n \implies \epsilon > \phi(x)$$

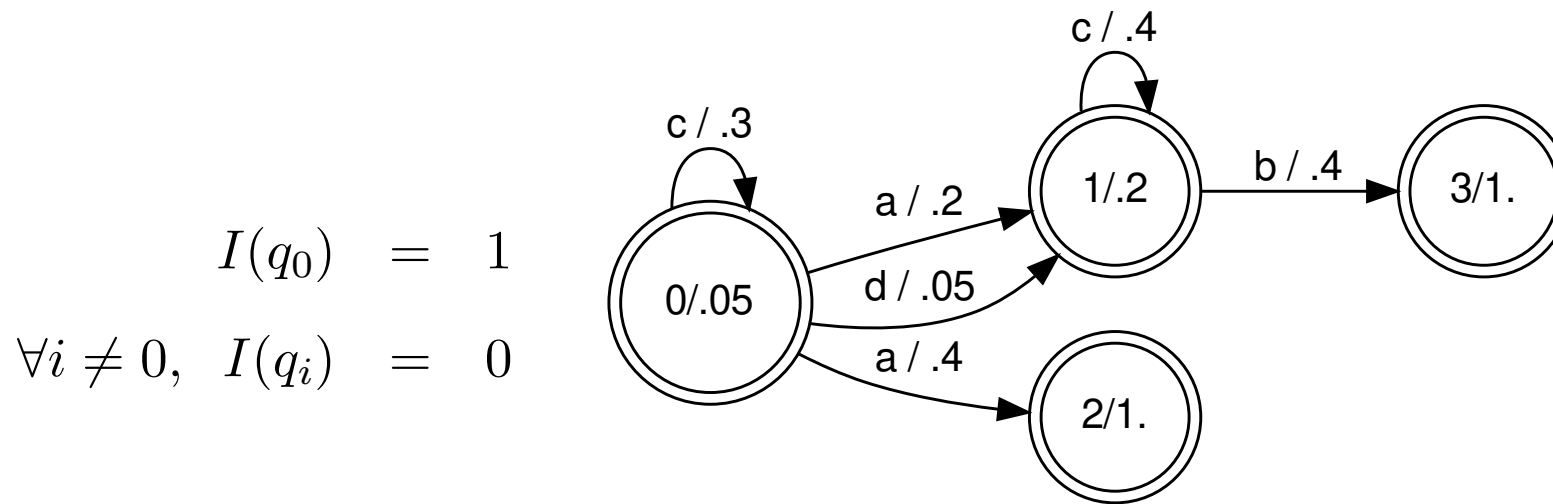
Problems	Models	
Non-structured	Generative	Naive Bayes classifier
	Discriminative	Logistic Regression Perceptron algorithm Support Vector Machines Neural networks
Structured	Generative	Probabilistic Finite State Automata Hidden Markov Models Probabilistic Context-Free Grammars
	Discriminative	Conditional Random Fields Structured Perceptron Structured Support Vector Machines (Encoder-Decoder) Recurrent Neural Network

Definition. A **probabilistic automaton** A over a probabilistic semiring is a tuple $A = (\Sigma, Q, \delta, I, F, P)$, where

- Σ is the alphabet;
- Q is a finite set of states;
- $\delta \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times Q$ is a set of transitions;
- $I : Q \rightarrow \mathbb{R}^+$ (initial-state probabilities);
- $F : Q \rightarrow \mathbb{R}^+$ (final-state probabilities);
- $P : \delta \rightarrow \mathbb{R}^+$ (transition probabilities).

I , F , and P are probabilistic functions that must satisfy:

$$\sum_{q \in Q} I(q) = 1,$$
$$\forall q \in Q \quad F(q) + \sum_{a \in \Sigma; q' \in Q} P(q, a, q') = 1.$$



$$P_A(acb, q_0q_1q_1q_3) = I(q_0) \cdot P(q_0, a, q_1) \cdot P(q_1, c, q_1) \cdot P(q_1, b, q_3) \cdot F(q_3) = 0,032$$

$$P_A(a, q_0q_1) = I(q_0) \cdot P(q_0, a, q_1) \cdot F(q_1) = 0,04$$

$$P_A(a, q_0q_2) = I(q_0) \cdot P(q_0, a, q_2) \cdot F(q_2) = 0,4$$

$$P_A(a) = P_A(a, q_0q_1) + P_A(a, q_0q_2) = 0,44$$

- The input string $x \in \Sigma^*$ will be accepted by the PFSA A , if there exists a path π (sequence of transitions), $s_0 s_1 \dots s_k$ ($|x| \leq k$), such that:

$$\pi = (s_0, x_1, s_1) \cdot (s_1, x_2, s_2) \cdots (s_{k-1}, x_k, s_k)$$

- The probability that the PFSA A , will accept the input string $x \in \Sigma^*$ through the path (sequence of transitions) π is:

$$P_A(x, \pi) = I(s_0) \cdot \left(\prod_{i=1}^k P(s_{i-1}, x_i, s_i) \right) \cdot F(s_k)$$

- The probability that the PFSA A , will accept the input string $x \in \Sigma^*$

$$P_A(x) = \sum_{\pi \in \Pi_A(x)} P_A(x, \pi)$$

➤ Probability of a path

$$P_A(x, \pi) = I(s_0) \cdot \left(\prod_{i=1}^k P(s_{i-1}, x_i, s_i) \right) \cdot F(s_k)$$

➤ Probability of a string

$$P_A(x) = \sum_{\pi \in \Pi_A(x)} P_A(x, \pi)$$

➤ Probability of the best path

$$\widehat{P}_A(x) = \max_{\pi \in \Pi_A(x)} P_A(x, \pi)$$

➤ Best path

$$\widehat{\pi}(x) = \arg \max_{\pi \in \Pi_A(x)} P_A(x, \pi)$$

➤ Language accepted by a probabilistic automaton

$$L(A) = \{x \in L(A) \mid P_A(x) > 0\}$$

- **Consistency of PFSA:** A probabilistic automaton A is consistent **iff**:

$$\sum_{x \in L(A)} P_A(x) = 1$$

- **Definition:** A state of a PFSA A is useful if it appears in at least one valid path of Π_A .
- **Proposition:** A PFSA is consistent if all its states are useful.

Definition. A **hidden Markov model (HMM)** M over a probabilistic semiring is a tuple $M = (\Sigma, Q, F, I, T, E)$, where

- Σ is the alphabet;
- Q is a finite set of states;
- $q_f \in F \subseteq Q$ is a special (final) state;
- $I : (Q - \{q_f\}) \rightarrow \mathbb{R}^+$ is an initial state probability function;
- $T : (Q - \{q_f\}) \times Q \rightarrow \mathbb{R}^+$ is a state state probability function;
- $E : (Q - \{q_f\}) \times \Sigma \rightarrow \mathbb{R}^+$ is a state-based symbol emission probability function.

I , T , and E are probabilistic functions that must satisfy:

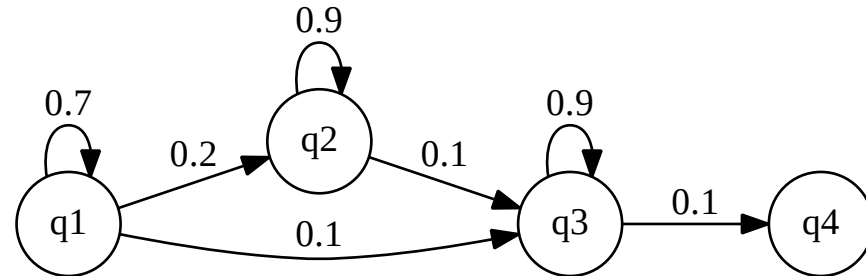
$$\begin{aligned} \sum_{q \in (Q - \{q_f\})} I(q) &= 1, \\ \forall q \in (Q - \{q_f\}) \quad \sum_{q' \in Q} T(q, q') &= 1 \\ \forall q \in (Q - \{q_f\}) \quad \sum_{a \in \Sigma} E(q, a) &= 1 \end{aligned}$$

Symbol emission probabilities

a	0.9	a	0.1	a	0.9
b	0.1	b	0.9	b	0.1

State transition probabilities

Initial state probabilities



$$I(q_1) = 1; \quad I(q_2) = I(q_3) = I(q_4) = 0$$

$$P_M(aba, q_1q_2q_3q_4) = I(q_1) E(a, q_1) T(q_1, q_2) E(b, q_2) T(q_2, q_3) E(a, q_3) T(q_3, q_4) = 1,458 \cdot 10^{-3}$$

$$P_M(aba, q_1q_1q_3q_4) = I(q_1) E(a, q_1) T(q_1, q_1) E(b, q_1) T(q_1, q_3) E(a, q_3) T(q_3, q_4) = 0,567 \cdot 10^{-3}$$

$$P_M(aba, q_1q_3q_3q_4) = I(q_1) E(a, q_1) T(q_1, q_3) E(b, q_3) T(q_3, q_3) E(a, q_3) T(q_3, q_4) = 0,729 \cdot 10^{-3}$$

$$P_M(aba) = 2,754 \cdot 10^{-3}$$

- The probability that the HMM M , will accept the input string $x = x_1 \dots x_k \in \Sigma^*$ through the path (sequence of transitions) $\pi = s_1 \dots s_k$ is:

$$P_M(x, \pi) = I(s_1) \cdot \prod_{i=2}^k T(s_{i-1}, s_i) \cdot \prod_{i=1}^k E(x_i, s_i)$$

- The probability that the HMM M , will accept the input string $x \in \Sigma^*$

$$P_M(x) = \sum_{\pi \in \Pi_A(x)} P_M(x, \pi)$$

- Given an HMM M , there exists a PFSA A , such that $P_M(x) = P_A(x) \quad \forall x \in \Sigma^*$

➤ Probabilistic Context-Free Grammars: $G_\theta = (G, P)$

➤ $G = (\Sigma, N, S, \mathcal{P})$ characteristic grammar

➤ $P : \mathcal{P} \rightarrow]0, 1]$ probabilities of rules:

$$\forall A_i \in N \quad P(A_i \rightarrow \alpha_j) \equiv P(r_{ij}) \equiv P(A_i \rightarrow \alpha_j \mid A_i) \equiv P(\alpha_j \mid A_i)$$

where n_i is the number of rules with A_i in the left side of rule:

$$\sum_{1 \leq j \leq n_i} P(\alpha_j \mid A_i) = 1$$

➤ Probabilistic derivation

Given a sequence of stochastic events:

$$S = \alpha_0 \xrightarrow{r_1} \alpha_1 \xrightarrow{r_2} \alpha_2 \cdots \alpha_{m-1} \xrightarrow{r_m} \alpha_m = x$$

the probability of x being generated by $G_\theta = (G, P)$ from the rule sequence

$t_x = r_1 \cdots r_m$, is:

$$P_\theta(x, t_x) = P(r_1) \cdot P(r_2 \mid r_1) \cdots P(r_m \mid r_1 \cdots r_{m-1})$$

➤ **problem:** computation of the probabilities

➤ **restriction:** $P(r_j \mid r_1 \cdots r_{j-1}) = P(r_j)$

$$P_\theta(x, t_x) = \prod_{j=1 \cdots m} P(r_j)$$

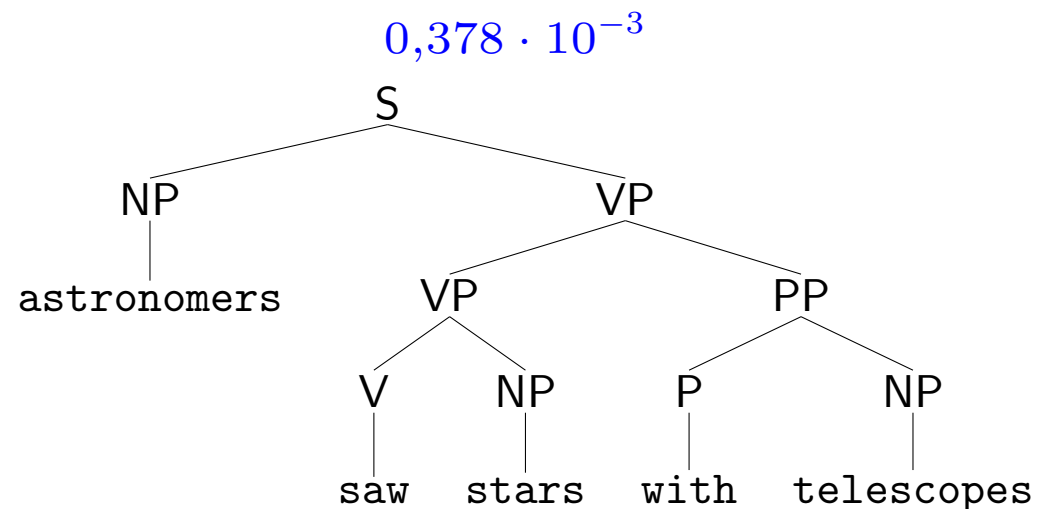
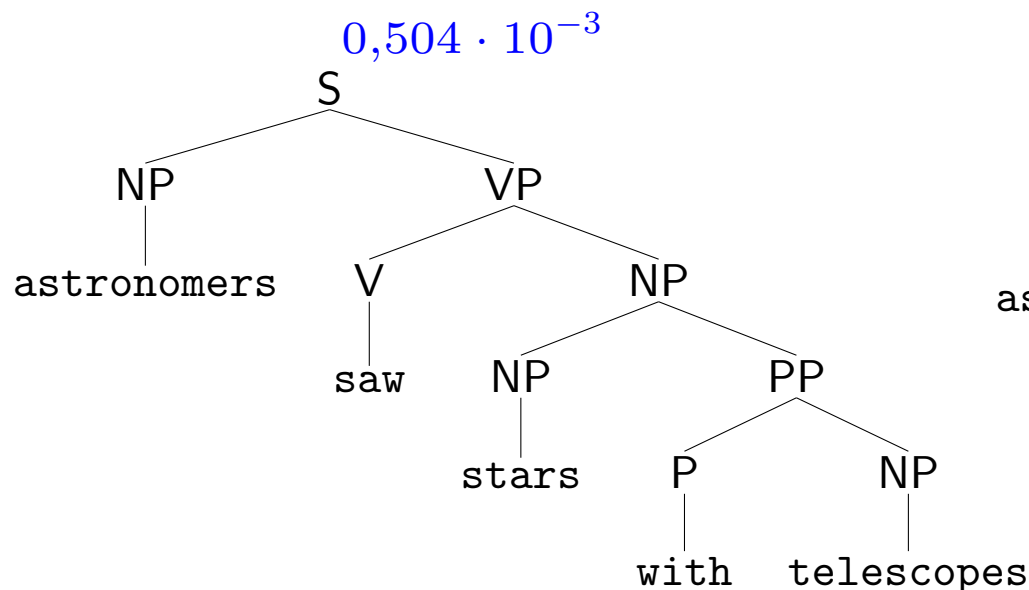
PROBABILISTIC CONTEXT-FREE GRAMMARS: EXAMPLE

Example: A simple Context-Free Grammars

[Manning and Schütze, 2002]

$S \rightarrow NP VP$	$1,0$	$VP \rightarrow V NP$	$0,7$	$V \rightarrow \text{saw}$	$1,0$	$NP \rightarrow \text{saw}$	$0,04$
$NP \rightarrow NP PP$	$0,4$	$VP \rightarrow VP PP$	$0,3$	$NP \rightarrow \text{astronomers}$	$0,1$	$NP \rightarrow \text{stars}$	$0,18$
$PP \rightarrow P NP$	$1,0$	$P \rightarrow \text{with}$	$1,0$	$NP \rightarrow \text{ears}$	$0,18$	$NP \rightarrow \text{telescopes}$	$0,1$

$S \xRightarrow{1,0} NP VP \xRightarrow{0,1} \text{astronomers} VP \xRightarrow{0,7} \text{astronomers} V NP \xRightarrow{1,0} \text{astronomers saw} NP \xRightarrow{0,4}$
 $\text{astronomers saw} NP PP \xRightarrow{0,18} \text{astronomers saw stars} PP \xRightarrow{1,0} \text{astronomers saw stars} P NP \xRightarrow{1,0}$
 $\text{astronomers saw stars with} NP \xRightarrow{0,1} \text{astronomers saw stars with telescopes} = 0,504 \cdot 10^{-3}$



- Probability of a parse tree

$$P_{\theta}(x, t_x) = \prod_{j=1 \dots m} P(r_j)$$

- Probability of a string

$$P_{\theta}(x) = \sum_{t_x \in \mathcal{T}_x} P_{\theta}(x, t_x)$$

- Probability of the best parse tree

$$\widehat{P}_{\theta}(x) = \max_{t_x \in \mathcal{T}_x} P_{\theta}(x, t_x)$$

- Language generated by a probabilistic grammar

$$L(G_{\theta}) = \{x \in L(G) \mid P_{\theta}(x) > 0\}$$

➤ Consistent grammars

A probabilistic grammar $G_\theta = (G, P)$ is consistent **iff**:

$$\sum_{x \in L(G)} P_\theta(x) = 1$$

➤ Theorem [Booth-Thompson,73]

There exist probabilistic languages (L, ϕ) that can not be generated by a probabilistic grammar $G_\theta = (G, P)$

➤ **Example.-** Let $L = \{a^n b^n \mid n \geq 0\}$ be a probabilistic language:

$$\phi(a^n b^n) = \frac{1}{e \, n!}$$

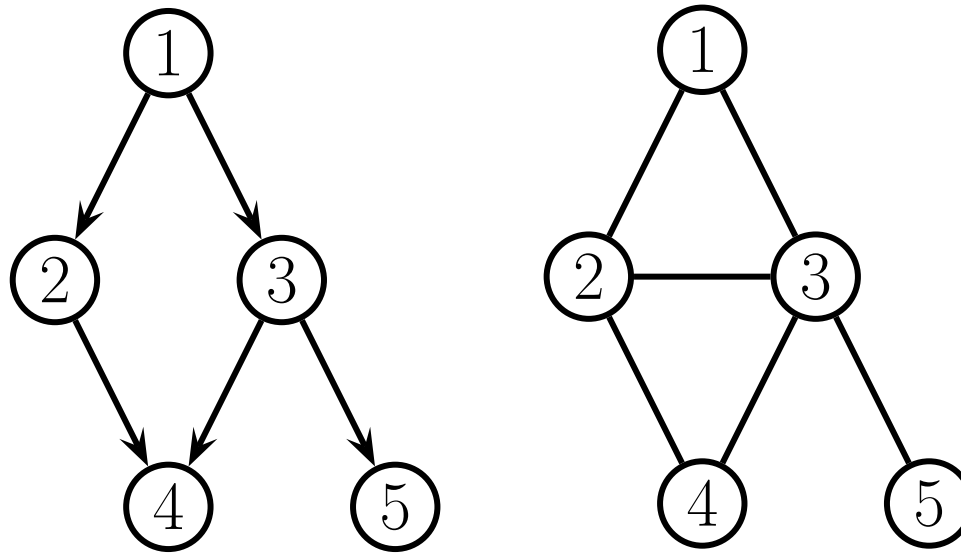
There is not any G_θ such that $\phi(x) = P_\theta(x) \quad \forall x \in L$

Probabilistic Graphical Models are an elegant framework that combines uncertainty (probabilities) and logical structure (independence constraints) to make simplifying assumptions about which variables affect which other variables.

These assumptions can often be represented graphically, leading to whole sets of models known collectively as **Graphical Models** (GMs)

Why do we need Graphical Models?

- GMs are the basis for the probabilistic approach to *Intelligent Systems*.
- GMs are a compact representation of joint probability distributions using graphs, constituting a perfect match between **Probability Theory** and **Graph Theory**.
- GMs allow us to abstract out the conditional independence relationships between the variables from the details of their parametric forms.
- GMs generalize to Neural Networks and Hidden Markov Models, among others.



[from K.P. Murphy, 2012]

Nodes represent random variables.

The **graph** represents a set of independences and factorizes a distribution.

Graphical models: taxonomy

- **Bayesian Networks** based on **Directed Graphs**.
- **Markov Random Fields** based on **Undirected Graphs**.

Key components

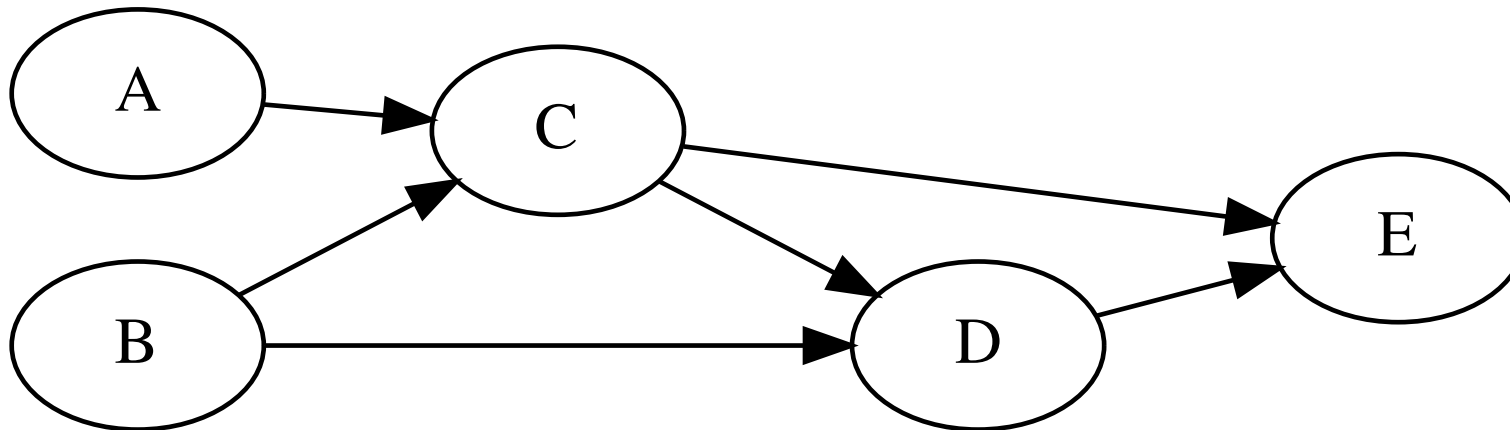
- **Inference:** deduce probability distributions from other given.
- **Learning:** obtain the probabilistic model from observations.

Applications

- Medical diagnosis, fault detection, ...
- **Computer Vision:** image segmentation, 3D reconstruction, scene analysis, ...
- **Natural Language Processing:** speech recognition, information extraction, machine translation, ...
- **Robotics:** planning, detection, ...

A **Bayesian Network** (BN) is a graphical model that represents a set of random variables and their conditional dependencies via a **Directed Acyclic Graph** (DAG).

Example



Nodes represent to random variables (discrete or continuous)

Edges represent statistical dependencies between the variables

➤ There are two extreme cases:

$$1) \quad p(x_1, x_2, \dots, x_T) = p(x_1) \cdot p(x_2|x_1) \dots p(x_T|x_1, \dots, x_{T-1})$$

$$= p(x_1) \prod_{t=2}^T p(x_t|x_1, \dots, x_{t-1}) \quad \text{requires } 2^T - 1 \text{ parameters}$$

$$2) \quad p(x_1, x_2, \dots, x_T) = \prod_{t=1}^T p(x_t) \quad \text{requires } T \text{ parameters}$$

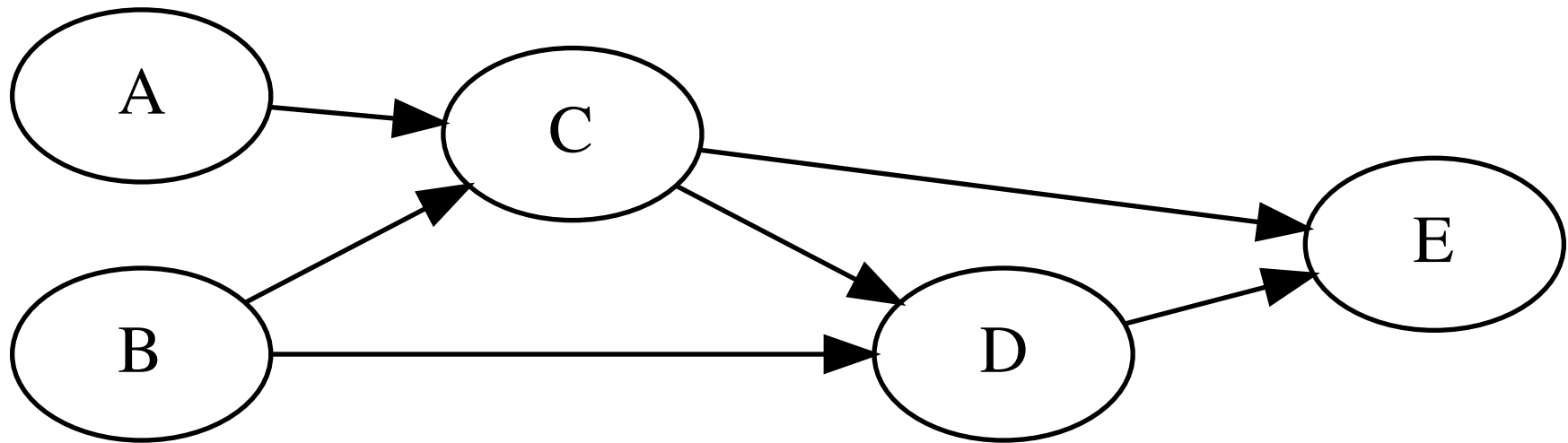
➤ If not all dependencies are possible, the complete factorization will be reflected in the graph (DAG) associated with the BN.

➤ A BN with nodes $x_1, \dots, x_T$ defines a joint probability distribution:

$$p(x_1, x_2, \dots, x_T) = \prod_{t=1}^T p(x_t | \varphi(x_t))$$

where $\varphi(x_t)$ denotes the dependencies associated with node x_t

Example



$$p(A, B, C, D, E) = p(A) \cdot p(B) \cdot p(C|A, B) \cdot p(D|B, C) \cdot p(E|C, D)$$

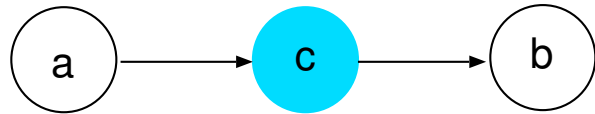
Representing knowledge through the notion of **conditional independence**

- Two events, a and b , are **(unconditionally) independent** and we denote it as $(a \perp b)$ if:

$$p(a, b) = p(a) \cdot p(b) \quad \text{or} \quad p(a | b) = p(a)$$

- Two events, a and b , are **conditionally independent** given an event c , and we denote it as $(a \perp b | c)$ if:

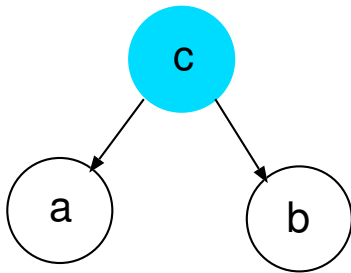
$$p(a | b, c) = p(a | c) \quad \text{or} \quad p(a, b | c) = p(a | c) \cdot p(b | c)$$



Causal direction: $(a \perp\!\!\!\perp b \mid c) \quad (a \not\perp\!\!\!\perp b)$

$$P(a, b \mid c) = \frac{P(a)P(c \mid a)P(b \mid c)}{P(c)} = P(a \mid c)P(b \mid c)$$

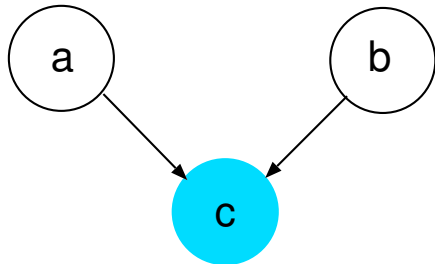
but $P(a, b) \neq P(a)P(b)$



Common parent: $(a \perp\!\!\!\perp b \mid c) \quad (a \not\perp\!\!\!\perp b)$

$$P(a, b \mid c) = \frac{P(c)P(a \mid c)P(b \mid c)}{P(c)} = P(a \mid c)P(b \mid c)$$

but $P(a, b) \neq P(a)P(b)$



V-structure: $(a \not\perp\!\!\!\perp b \mid c) \quad (a \perp\!\!\!\perp b)$

$$P(a, b \mid c) \neq P(a \mid c)P(b \mid c)$$

but $P(a, b) = \sum_c P(a)P(b)P(c \mid a, b) = P(a)P(b)$

The purpose is to estimate the posterior probability of some variable x from the joint distributions associated with a BN, given some evidence y (regardless of the values of the rest of the variables z).

$$P(x \mid y) = \frac{P(x, y)}{P(y)} \quad \text{con:} \quad P(x, y) = \sum_z P(x, y, z); \quad P(y) = \sum_{x, z} P(x, y, z);$$

The goal is to efficiently calculate $P(x, y)$ and $P(y)$.

➤ The usefulness of calculating the posterior probability.

Prediction.- What is the probability of observing a symptom knowing that the patient has a particular disease?

Diagnosis.- What is the probability that a particular disease is a correct diagnosis given some symptoms?

$P(S \mid R)$			$P(R)$		
Rain (R)		Sprinkler (S)	Rain (R)		
		s r	n y		
n		0.60 0.40	0.8	0.2	
y		0.99 0.01			


```

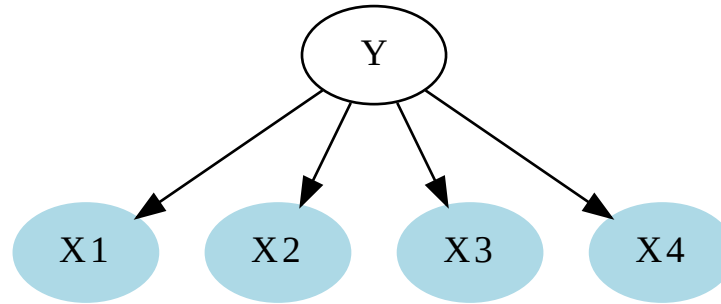
graph LR
    Rain((Rain)) --> Sprinkler((Sprinkler))
    Rain --> Grass((Grass))
    Sprinkler --> Grass
    
```


$P(G \mid S, R)$		
Rain (R) Sprinkler (S)		Grass (G)
		d w
n	s	0.99 0.01
n	r	0.10 0.90
y	s	0.20 0.80
y	r	0.01 0.99

Joint distribution: $P(R, S, G) = P(R) P(S \mid R) P(G \mid R, S)$

Exercise: What is the probability that the sprinkler will work if there is no rain and the grass is wet?

Classifiers

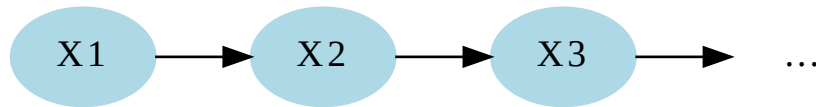


Naive Bayes

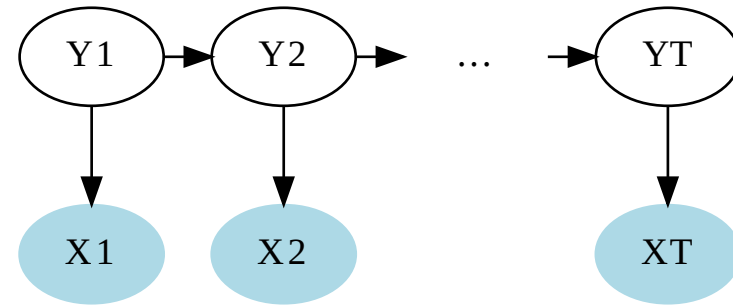
- Predicting a single class y given a vector of features $\mathbf{x} = \{x_1, x_2, \dots, x_T\}$
- Assumption: once the class label is known all the features are independent

$$p(x_1, x_2, \dots, x_T, y) = p(y) \prod_{t=1}^T p(x_t|y)$$

Sequential Prediction



Sequence Labeling



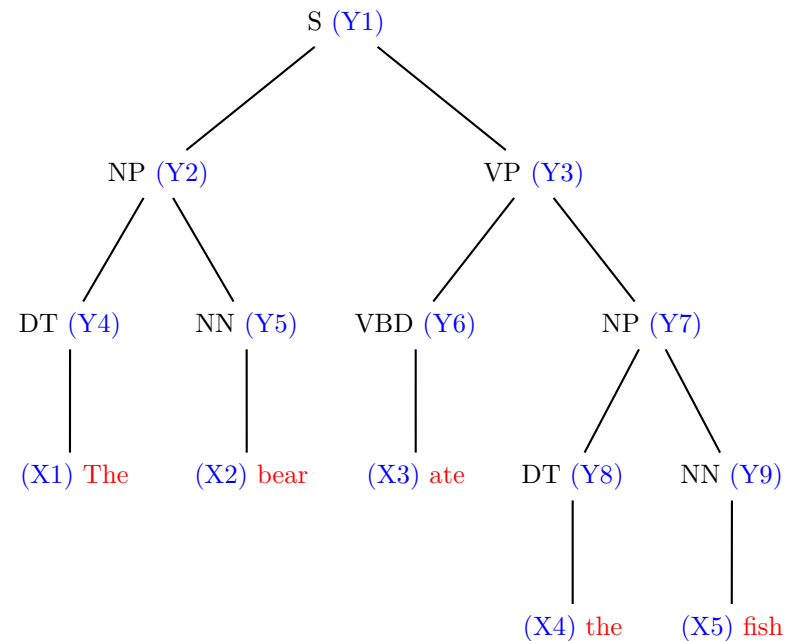
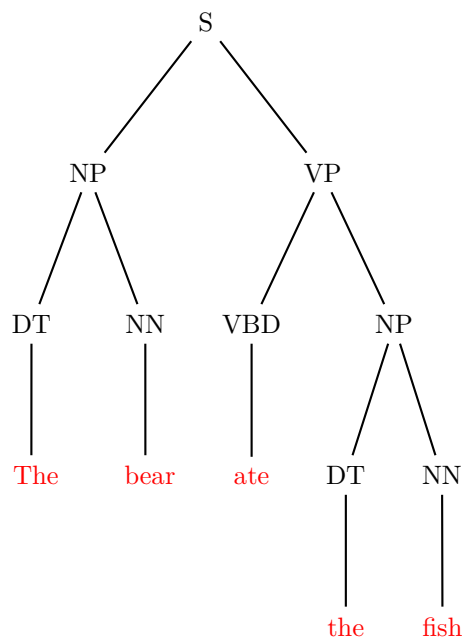
Markov model

$$p(x_1, x_2, \dots, x_T) = p(x_1) \cdot p(x_2|x_1) \cdot p(x_3|x_2) \dots = p(x_1) \prod_{t=1}^T p(x_t|x_{t-1})$$

(Bigram) Hidden Markov Model

$$P(x_1^T, y_1^T) = \prod_{t=1}^T q(y_t|y_{t-1}) \cdot e(x_t|y_t)$$

Parsing



Probabilistic Context-Free Grammars

$$t_x = [S = \alpha_0 \xRightarrow{r_1} \alpha_1 \xRightarrow{r_2} \alpha_2 \cdots \alpha_{m-1} \xRightarrow{r_m} \alpha_m = \mathbf{x}] \quad P(x_1^T, t_x) = \prod_{i=1 \cdots m} P(r_i);$$

$$P(r_i) = P(Y_j \ Y_k | Y_i) \quad \text{or} \quad P(r_i) = P(X_l | Y_i)$$

Bayesian Networks cannot perfectly represent all distributions.

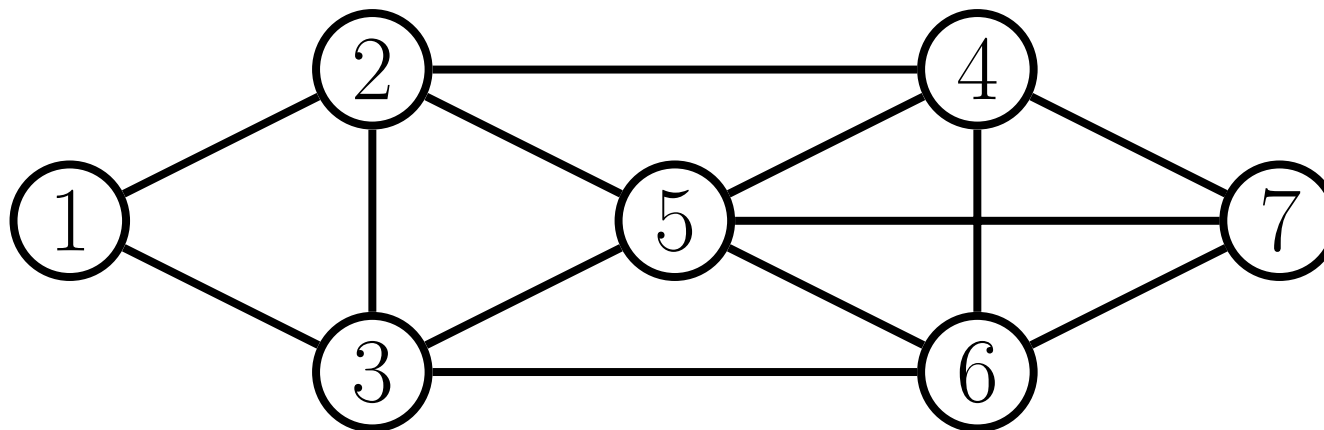
Markov Random Fields (MRF) are graphical models based on Undirected Graphs.

The **nodes** represent the random variables.

The **edges** represent some notion of probabilistic interaction between neighboring nodes. Edges show which variables depend on each other.

Example

[Kevin P. Murphy, 2012]

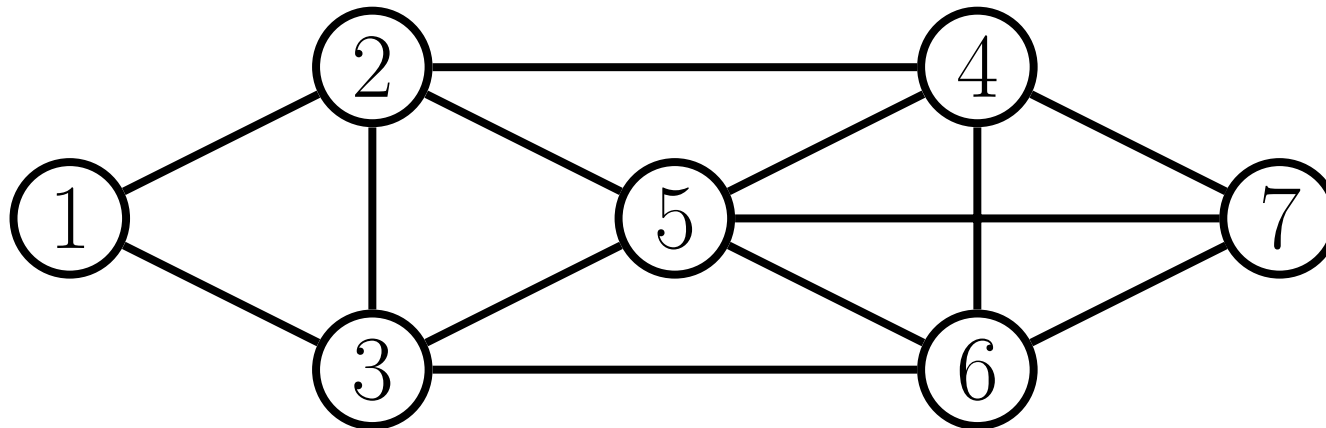


Conditional independence properties: **global Markov property** for MRFs

MRFs define conditional independence relationships via simple graph separation:
for sets of nodes A , B , and C , we say,

$$A \perp B \mid C \Leftrightarrow C \text{ separates } A \text{ from } B \text{ in the graph.}$$

Example:

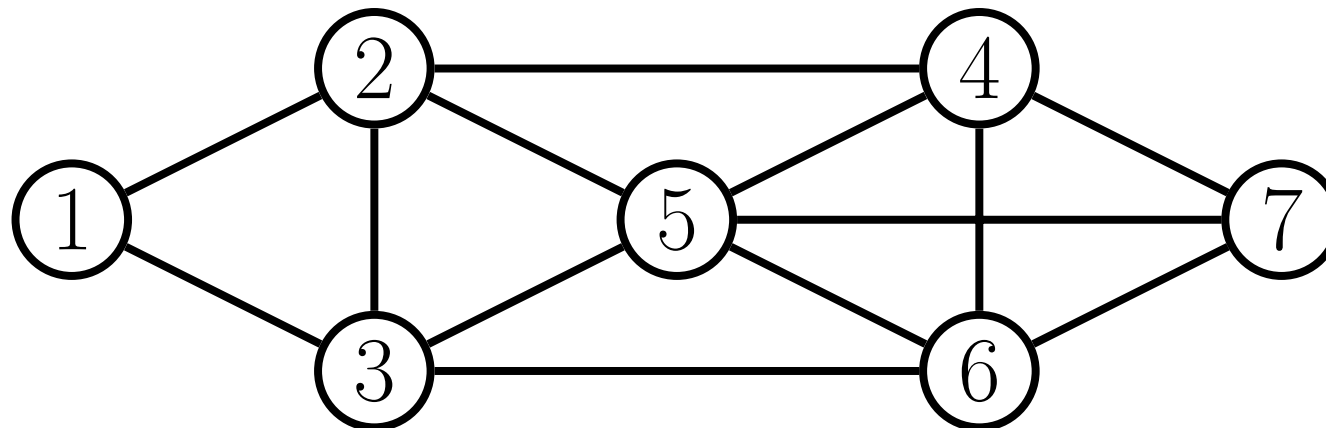


$$\{1, 2\} \perp \{6, 7\} \mid \{3, 4, 5\}$$

Definitions: A **clique** of an undirected graph is a fully connected subgraph.

A **maximal clique** is a clique that is not a subgraph of any other clique.

Example:



Cliques: $\{1, 2\}; \dots \{1, 2, 3\}; \{2, 3, 5\}; \{2, 4, 5\}; \{3, 5, 6\}; \{4, 5, 6\} \dots \{4, 5, 6, 7\};$

No-cliques: $\{1, 2, 3, 5\}; \{2, 3, 4, 5, 6\}; \dots$

maximal cliques: $\{1, 2, 3\}; \{2, 3, 5\}; \{2, 4, 5\}; \{3, 5, 6\}; \{4, 5, 6, 7\};$

Definition.- Let G be the undirected graph that underlies an MRF. A probability distribution P_G defines a factorization over G if it is associated with:

$$P_G(X_1, \dots, X_T) = \frac{1}{Z} \prod_{C \in Q} \psi_C(V_C)$$

Where,

$V = \{X_1, X_2, \dots, X_T\}$ is the set of random variables;

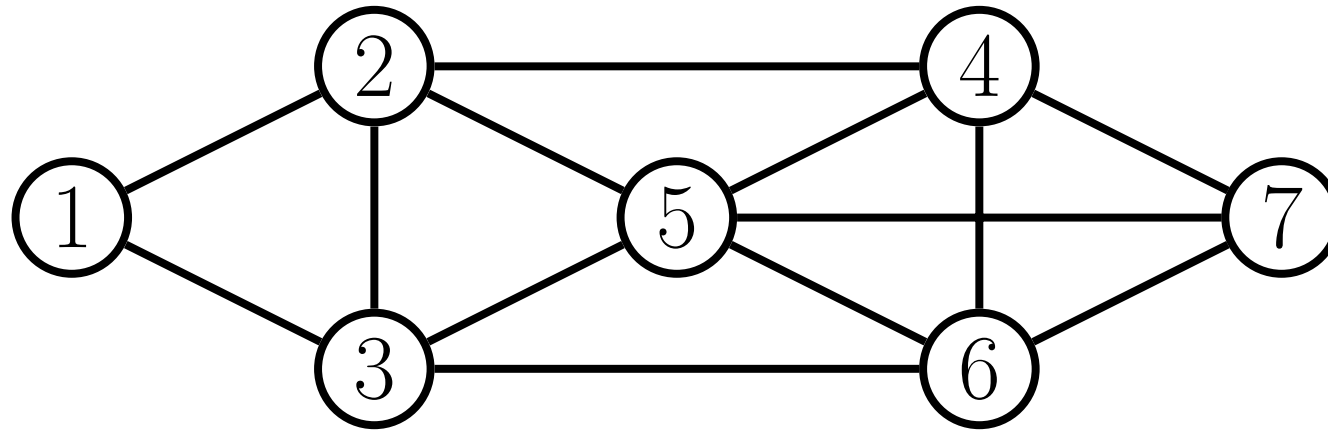
Q is the set of all the (maximal) cliques of G ;

V_C is the subset of variables from clique C ;

$\psi_C : Q \rightarrow \mathbb{R}^{>0}$ is the **potential function** for clique c , and

Z is a normalization factor (constant) defined as:

$$Z = \sum_{X_1, \dots, X_T} \prod_{C \in Q} \psi_C(V_C)$$



Example $Q = \{ \{1, 2, 3\}; \{2, 3, 5\}; \{2, 4, 5\}; \{3, 5, 6\}; \{4, 5, 6, 7\}; \}$

$$P_G(1, 2, 3, 4, 5, 6, 7) = \frac{1}{Z} \psi_1(1, 2, 3) \cdot \psi_2(2, 3, 5) \cdot \psi_3(2, 4, 5) \cdot \psi_4(3, 5, 6) \cdot \psi_5(4, 5, 6, 7)$$

MRFs are more powerful than BNs but are more challenging to deal with computationally.

It is often helpful to specify these potential functions by using exponential transformations,

$$\begin{aligned} P_G(X_1, \dots, X_T) &= \frac{1}{Z} \prod_{C \in Q} \psi_C(V_C) \\ &= \frac{1}{Z} \prod_{C \in Q} \exp(-E_C(V_C)) \\ &= \frac{1}{Z} \exp\left(-\sum_{C \in Q} E_C(V_C)\right) \end{aligned}$$

Where $E_C : Q \rightarrow \mathbb{R}$ is a function, which is called the **energy function**.

There are types of energy functions that can be defined by *generalized linear functions*:

$$E_C(V_C) = - \sum_k \theta_{C,k} f_{C,k}(V_C)$$

Given $x = x_1, x_2, \dots, x_T \in \mathcal{X}^*$ and $y = y_1, y_2, \dots, y_T \in \mathcal{Y}^*$

➤ Discriminative models: **Conditional Random Fields (CRF)**

[Lafferty, McCallum, Pereira, 2001].

➤ Model parameters θ

➤ Compatibility function $\phi(x, y; \theta) \rightarrow \mathcal{R}$

that that gives high positive scores to compatibles pairs (x, y)

➤ Conditional distribution:

$$p(y | x; \theta) = \frac{\exp\{\phi(x, y; \theta)\}}{Z(x; \theta)}$$

Where

$$Z(x; \theta) = \sum_{y' \in \mathcal{Y}^*} \exp\{\phi(x, y'; \theta)\}$$

Compatibility Functions from (Bigram) Indicator Features

Example Named-Entity Recognition

[C.Sutton and A.McCallum, 2012]

Entities = { (P) people, (O) organizations, (L) locations, (M) other }

$\mathcal{Y} = \{ \text{B-P, I-P, B-O, I-O, B-L, I-L, B-M, I-M, O} \}$ [CoNLL 2003 data set]

U.N.	official	Ekeus	heads	for	Baghdad
B-O	O	B-P	O	O	B-L

➤ Label-label features

$$f_{ij}^{LL}(y_{t-1}, y_t, x_t) = \begin{cases} 1 & \text{if } y_{t-1} = i \text{ and } y_t = j \quad \forall i, j \in \mathcal{Y} \\ 0 & \text{otherwise} \end{cases}$$

There are 9 different labels, so there are 81 label-label features.

➤ Label-word features

$$f_{iv}^{LW}(y_{t-1}, y_t, x_t) = \begin{cases} 1 & \text{if } y_t = i \text{ and } x_t = v \quad \forall i \in \mathcal{Y}, \forall v \in \mathcal{V} \\ 0 & \text{otherwise} \end{cases}$$

$\mathcal{V}$ is the set of all corpus words. For the CoNLL 2003 data set, $|\mathcal{V}| = 21,249$ words so there are 191,241 label-word features.

➤ Label-observation features

$$f_{ib}^{LO}(y_{t-1}, y_t, x_t) = \begin{cases} 1 & \text{if } y_t = i \text{ and } g_b(x_t) \quad \forall i \in \mathcal{Y} \\ 0 & \text{otherwise} \end{cases}$$

All observation functions are binary functions and are heuristically defined depending on the task

➤ Let $f(y_{i-1}, y_i, x_i)$ be a vector of features

$$(f_1(y_{i-1}, y_i, x_i), f_2(y_{i-1}, y_i, x_i), \dots, f_K(y_{i-1}, y_i, x_i))$$

➤ Given $f(\cdot)$ and let $\theta \in \mathcal{R}^K$ be a weight matrix

$$\begin{aligned}\phi(x, y; \theta) &= \sum_{t=1}^T \theta f(y_{t-1}, y_t, x_t) \\ &= \sum_{t=1}^T \sum_{k=1}^K \theta_k f_k(y_{t-1}, y_t, x_t)\end{aligned}\tag{1}$$

➤ Let $y_0 = \text{NULL}$, so the bigram $y_0 y_1$ is defined for $t = 1$.

➤ This factorization will allow efficient algorithms since
if $y \neq y'$ share bigrams then they will share scores

$$\begin{aligned} p(y|x; \theta) &= \frac{\exp\{ \phi(x, y; \theta) \}}{Z(x; \theta)} \\ &= \frac{\exp\{ \sum_{t=1}^T \sum_{k=1}^K \theta_k f_k(y_{t-1}, y_t, x_t) \}}{Z(x; \theta)} \end{aligned} \quad (2)$$

Where

$$Z(x; \theta) = \sum_{y' \in \mathcal{Y}^*} \exp \left\{ \sum_{t=1}^T \sum_{k=1}^K \theta_k f_k(y'_{t-1}, y'_t, x_t) \right\} \quad (3)$$

- Features $f(\cdot)$ are given (problem-dependent).
- $\theta \in \mathcal{R}^K$ are the parameters of the model.
- CRFs are log-linear models on the feature functions.

- **Inference**: Compute the probability of an output sequence y for x

$$p(y \mid x; \theta) = \frac{1}{Z(x; \theta)} \prod_{t=1}^T \exp \left\{ \sum_{k=1}^K \theta_k f_k(y_{t-1}, y_t, x_t) \right\}$$

- **Decoding**: predict the best output sequence for x

$$\arg \max_{y \in \mathcal{Y}^*} p(y \mid x; \theta) = \arg \max_y \prod_{t=1}^T \exp \left\{ \sum_{k=1}^K \theta_k f_k(y_{t-1}, y_t, x_t) \right\}$$

- **Parameter estimation**: learn parameters θ , given training data

$$\{ (x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(N)}, y^{(N)}) \}$$

➤ Probabilistic Context-Free Grammars

- T.L.Booth and R.A.Thompson: *Applying Probability Measures to Abstract Languages*. IEEE Trans. on Compututers, 22(5):442–450. May 1973.
- C.S.Wetherell: *Probabilistic Languages: A Review and Some Open Questions*. ACM Computing Surveys, 12(4):361–379. December 1980. doi:10.1145/356827.356829.
- M.Collins: *Probabilistic Context-Free Grammars*. Course notes, Columbia University, 2011.

➤ Graphical Models

- C.M.Bishop: *Pattern Recognition and Machine Learning*. Chap.8: Graphical Models. Springer, 2006.
- D.Koller, N.Friedman, L.Getoor and B.Taskar: *Graphical Models in a Nutshell* in *An Introduction to Statistical Relational Learning*. MIT Press, 2007.

➤ Conditional Random Fields

- J.Lafferty, A.McCallum and F.C.N.Pereira: *Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data*. ICMI:282-289. 2001.
- C.Sutton and A.McCallum: *An introduction to conditional random fields for relational learning* in *An Introduction to Statistical Relational Learning*. MIT Press, 2007.

3. Making Prediction: Decoding and Inference

3.1. Linear generative models: $\Rightarrow$ PFSA

- Decoding: **Viterbi**,
- Inference: **Forward**

3.2. Linear discriminative models: $\Rightarrow$ CRFs

- Decoding: **Viterbi**,
- Inference: **Forward**

3.3. Non-linear generative models: $\Rightarrow$ PCFGs

- Decoding: probability of most likely parse tree: **Viterbi**
- Inference: probability of a string: **Inside**

3.4. Word Graph based N-best search

- Word graphs
- N-best search

➤ Probability of a path

$$P_A(x, \pi) = I(s_0) \cdot \left(\prod_{i=1}^k P(s_{i-1}, x_i, s_i) \right) \cdot F(s_k)$$

➤ Probability of a string

$$P_A(x) = \sum_{\pi \in \Pi_A(x)} P_A(x, \pi)$$

➤ Probability of the best path

$$\widehat{P}_A(x) = \max_{\pi \in \Pi_A(x)} P_A(x, \pi)$$

➤ Best path

$$\widehat{\pi}(x) = \arg \max_{\pi \in \Pi_A(x)} P_A(x, \pi)$$

- **Definition:** The probability of generating the prefix x_1^t through the best path and reaching state q is:

$$\gamma_x(t, q) \stackrel{\text{def}}{=} \max_{\substack{s_0^t \in \Pi_A(x_1^t): \\ s_t = q}} I(s_0) \cdot \left(\prod_{i=1}^t P(s_{i-1}, x_i, s_i) \right) \quad \forall q \in Q, 0 \leq t \leq |x|$$

- **Initialization:** $\gamma_x(0, q) = I(q) \quad \forall q \in Q$

- **Recursion:** $\forall q \in Q, 1 \leq t \leq |x|$

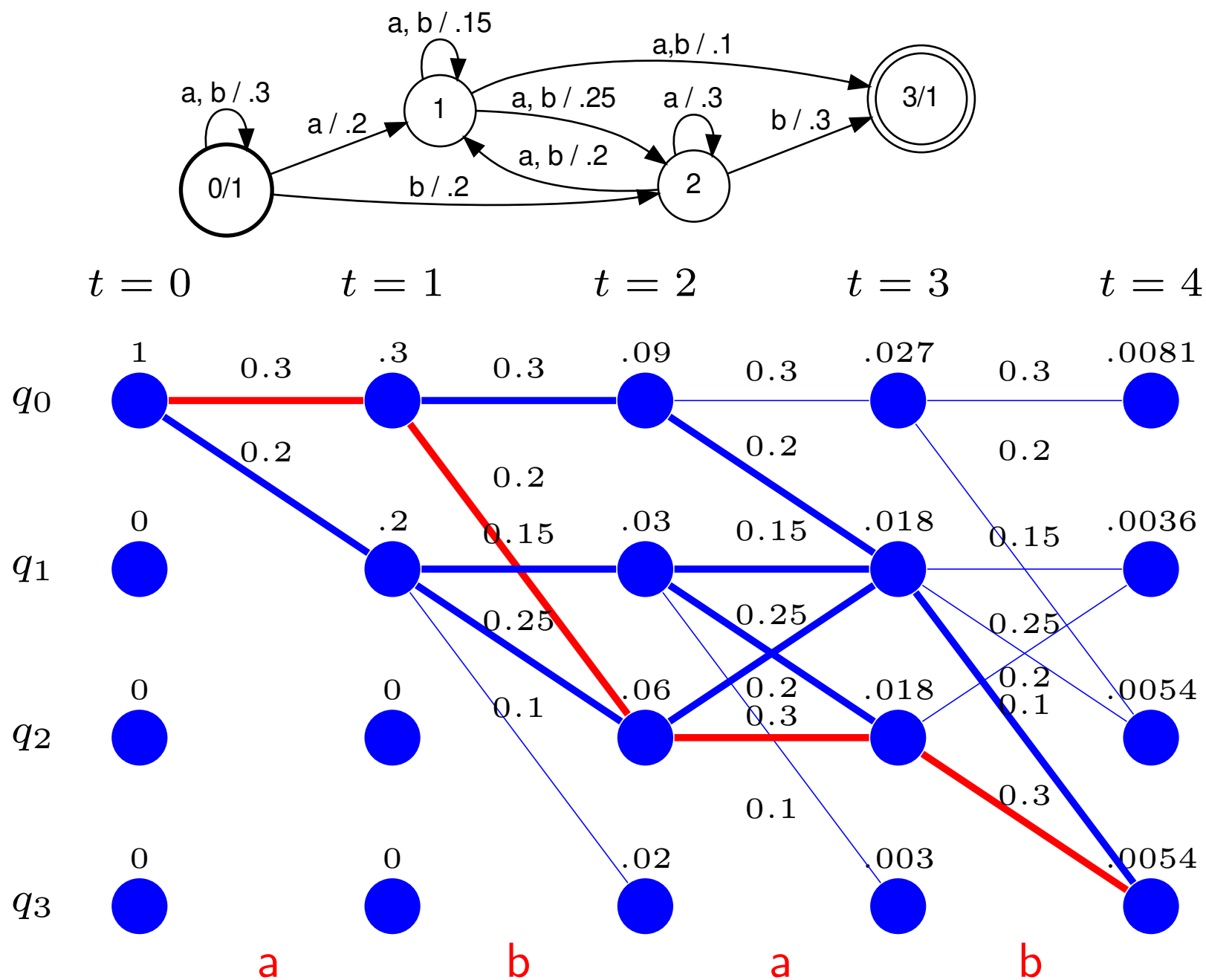
$$\gamma_x(t, q) = \max_{q' \in Q} \{ \gamma_x(t-1, q') \cdot P(q', x_t, q) \}$$

- **Final result:** The probability of the best state sequence for x given A is:

$$\hat{P}_A(x) = \max_{q \in Q} \{ \gamma_x(|x|, q) \cdot F(q) \}$$

Backpointers can be used to recover the optimal sequence of states

ALGORITMO DE VITERBI: EJEMPLO



➤ **Definition:** The probability of generating the prefix x_1^t and reaching state q is:

$$\alpha_x(t, q) \stackrel{\text{def}}{=} \sum_{\substack{s_0^t \in \Pi_A(x_1^t): \\ s_t = q}} I(s_0) \cdot \left(\prod_{i=1}^t P(s_{i-1}, x_i, s_i) \right) \quad \forall q \in Q, 0 \leq t \leq |x|$$

➤ **Initialization:** $\alpha_x(0, q) = I(q) \quad \forall q \in Q$

➤ **Recursion:** $\forall q \in Q, 1 \leq t \leq |x|$

$$\alpha_x(t, q) = \sum_{q' \in Q} \alpha_x(t-1, q') \cdot P(q', x_t, q)$$

➤ **Final result:** The probability of string x is:

$$P_A(x) = \sum_{q \in Q} \alpha_x(|x|, q) \cdot F(q)$$

➤ **Definition:** The probability of generating the suffix $x_{t+1}^{|x|}$ from the state q is:

$$\beta_x(t, q) \stackrel{\text{def}}{=} \sum_{\substack{s_t^{|x|} \in \Pi_A(x_{t+1}^{|x|}): \\ s_t = q}} \left(\prod_{i=t+1}^{|x|} P(s_{i-1}, x_i, s_i) \right) \cdot F(s_{|x|}) \quad \forall q \in Q, 0 \leq t \leq |x|$$

➤ **Initialization:** $\beta_x(|x|, q) = F(q) \quad \forall q \in Q$

➤ **Recursion:** $\forall q \in Q, 0 \leq t \leq |x| - 1$

$$\beta_x(t, q) = \sum_{q' \in Q} \beta_x(t+1, q') \cdot P(q, x_{t+1}, q')$$

➤ **Final result:** The probability of string x is:

$$P_A(x) = \sum_{q \in Q} I(q) \cdot \beta_x(0, q)$$

Given $x = x_1, x_2, \dots, x_T \in \mathcal{X}^*$ and $y = y_1, y_2, \dots, y_T \in \mathcal{Y}^*$

Discriminative models: **Conditional Random Fields (CRF)**

[Lafferty, McCallum, Pereira, 2001].

$$p(y|x; \theta) = \frac{\exp\{ \sum_{t=1}^T \sum_{k=1}^K \theta_k f_k(y_{t-1}, y_t, x_t) \}}{Z(x; \theta)} \quad (4)$$

Where

$$Z(x; \theta) = \sum_{y' \in \mathcal{Y}^*} \exp \left\{ \sum_{t=1}^T \sum_{k=1}^K \theta_k f_k(y'_{t-1}, y'_t, x_t) \right\} \quad (5)$$

We can define a transition (factor) as:

$$\Psi_t(y_{t-1}, y_t, x_t) \stackrel{\text{def}}{=} \exp \left\{ \sum_{k=1}^K \theta_k f_k(y_{t-1}, y_t, x_t) \right\}$$

From (4) CRFs can be defined as:

$$p(y|x; \theta) = \frac{1}{Z(x; \theta)} \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t)$$

Also from (5)

$$Z(x; \theta) = \sum_{y' \in \mathcal{Y}^*} \prod_{t=1}^T \Psi_t(y'_{t-1}, y'_t, x_t)$$

- **Decoding:** Given θ and x , predict the best output sequence for x .

$$\hat{y} = \arg \max_y p(y|x; \theta) = \arg \max_y \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t)$$

- The probability of an output sequence

$$p(y|x; \theta) = \frac{1}{Z(x; \theta)} \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t)$$

➤ **Definition:** Score of optimal sequence for $x_1 \dots x_t$ ending at $y_t = s \in \mathcal{Y}$

$$\gamma_t(s) \stackrel{\text{def}}{=} \max_{y_1^t; y_t=s} \prod_{i=1}^t \Psi_i(y_{i-1}, y_i, x_i)$$

➤ **Initialization:** $\forall s \in \mathcal{Y}$

$$\gamma_1(s) = \Psi_1(y_0 = \text{null}, y_1 = s, x_1)$$

➤ **Recursion:** $\forall t = 2 \dots T$; and $\forall s \in \mathcal{Y}$

$$\gamma_t(s) = \max_{s' \in \mathcal{Y}} \left\{ \gamma_{t-1}(s') \cdot \Psi_t(y_{t-1} = s', y_t = s, x_t) \right\}$$

➤ **Final result:** The optimal score for x is:

$$\max_s \gamma_T(s)$$

Backpointers can be used to recover the optimal sequence $\hat{y}$

➤ **Definition:** Score for $x_1 \dots x_t$ ending at $y_t = s \in \mathcal{Y}$

$$\alpha_t(s) \stackrel{\text{def}}{=} \sum_{y_1^t; y_t=s} \prod_{i=1}^t \Psi_i(y_{i-1}, y_i, x_i)$$

➤ **Initialization:** $\forall s \in \mathcal{Y}$

$$\alpha_1(s) = \Psi_1(y_0 = \text{null}, y_1 = s, x_1)$$

➤ **Recursion:** $\forall t = 2 \dots T$; and $\forall s \in \mathcal{Y}$

$$\alpha_t(s) = \sum_{s' \in \mathcal{Y}} \alpha_{t-1}(s') \cdot \Psi_t(y_{t-1} = s', y_t = s, x_t)$$

➤ **Final result:** The optimal score for x is:

$$\sum_s \alpha_T(s)$$

- The probability of an output sequence

$$p(y|x; \theta) = \frac{1}{Z(x; \theta)} \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t)$$

Where we can compute $Z(x; \theta)$ efficiently using the forward algorithm

$$Z(x; \theta) = \sum_{y_1'^T} \prod_{t=1}^T \Psi_t(y_{t-1}', y_t', x_t) = \sum_s \alpha_T(s)$$

Generative models: Probabilistic Context-Free Grammars

$$P_{\theta}(x, t_x) = \prod_{i=1}^m p(r_i) \quad \text{where } t_x : r_1, \dots, r_m$$

➤ Probability of an observation

$$P_{\theta}(x) = \sum_{t_x \in \mathcal{T}_x} P_{\theta}(x, t_x)$$

➤ Probability of the best parse tree

$$\hat{P}_{\theta}(x) = \max_{t_x \in \mathcal{T}_x} P_{\theta}(x, t_x)$$

➤ Finding the best parse

$$\hat{t}_x = \arg \max_{t_x \in \mathcal{T}_x} P_{\theta}(x, t_x)$$

➤ **Definition:** Given $x = x_1 \dots x_T \in \Sigma^*$ and $A \in N$

$$\widehat{e}(A, i, i + l) \stackrel{\text{def}}{=} \widehat{P}_\theta(A \xRightarrow{*} x_{i+1} \dots x_{i+l})$$

➤ **Initialization:** $\forall A \in N; \quad \forall i : 0 \dots T - 1;$

$$\widehat{e}(A, i, i + 1) = p(A \rightarrow b) \cdot \delta(b, x_{i+1})$$

➤ **Recursion:** $\forall A \in N; \quad \forall l : 2 \dots T; \quad \forall i : 0 \dots T - l;$

$$\widehat{e}(A, i, i + l) = \max_{B, C \in N} \left\{ p(A \rightarrow BC) \cdot \max_{k=1, \dots, l-1} \left\{ \widehat{e}(B, i, i + k) \cdot \widehat{e}(C, i + k, i + l) \right\} \right\}$$

➤ **Final result:** The probability of the best parse tree is:

$$\widehat{P}_\theta(x) = \widehat{e}(S, 0, T)$$

Backpointers can be used to recover the optimal sequence $\widehat{y}$

➤ **Definition:** Given $x = x_1 \dots x_T \in \Sigma^*$ and $A \in N$

$$e(A, i, i + l) \stackrel{\text{def}}{=} P_{\theta}(A \overset{*}{\Rightarrow} x_{i+1} \dots x_{i+l})$$

➤ **Initialization:** $\forall A \in N; \quad \forall i : 0 \dots T - 1;$

$$e(A, i, i + 1) = p(A \rightarrow b) \cdot \delta(b, x_{i+1})$$

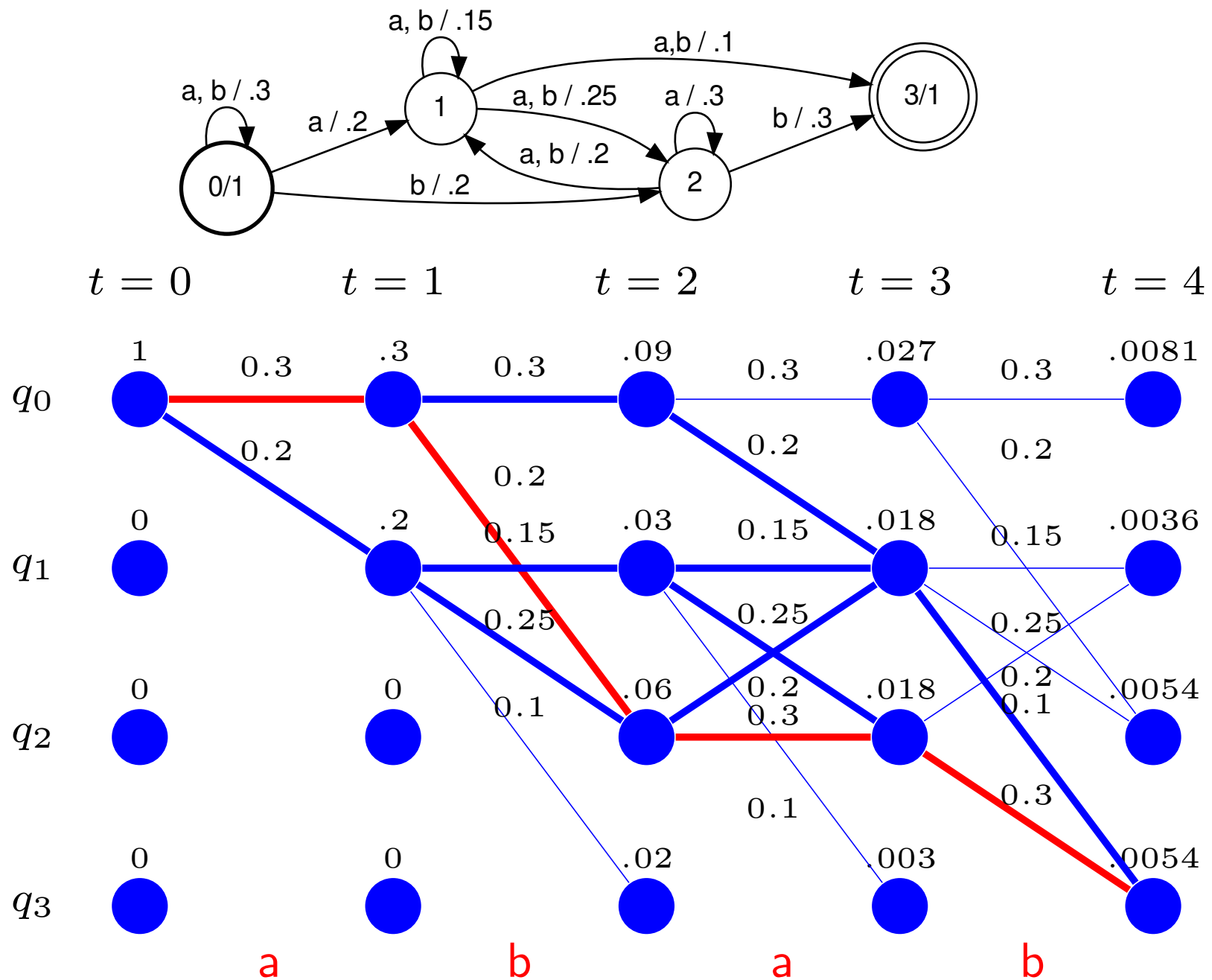
➤ **Recursion:** $\forall A \in N; \quad \forall l : 2 \dots T; \quad \forall i : 0 \dots T - l;$

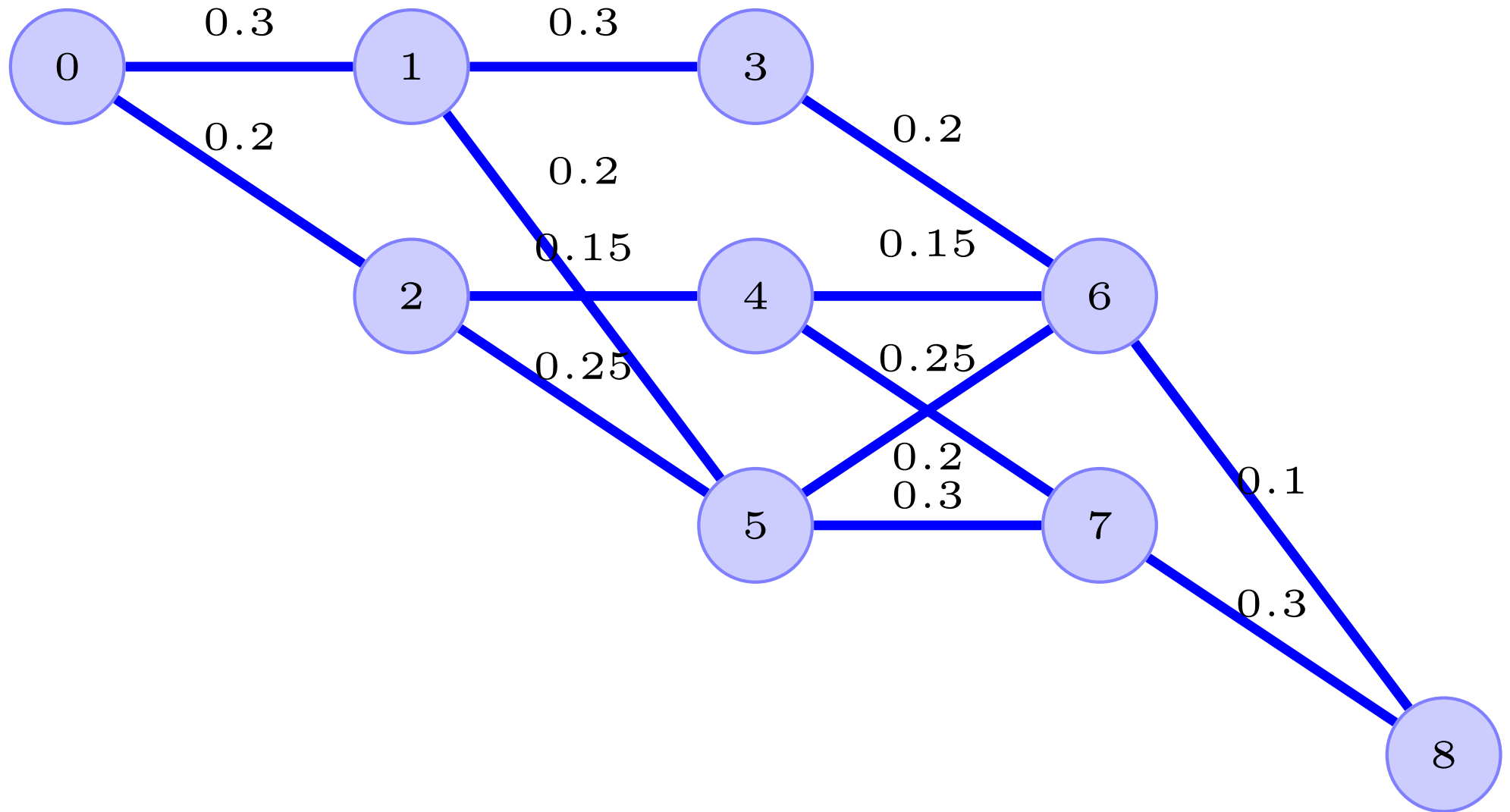
$$e(A, i, i + l) = \sum_{B, C \in N} \left\{ p(A \rightarrow BC) \cdot \sum_{k=1, \dots, l-1} e(B, i, i + k) \cdot e(C, i + k, i + l) \right\}$$

➤ **Final result:** The sentence probability is:

$$P_{\theta}(x) = e(S, 0, T)$$

WORD GRAPH BASED N-BEST SEARCH





➤ Why do we need word graphs?

- Trellis-based decoding is very expensive in time and space, even though pruning techniques were used.
- The Word Graph (WG) allow for a reduction of the search space to the most probable hypotheses.

➤ Word Graph (WG) definition

- A WG is a labeled weighted directed acyclic graph.
- A WG is a data structure that represents a large finite sample of word sequences very efficiently.
- A WG represents the interpretations with the highest posterior probabilities.
- The WG is (a pruned version of) the Viterbi search trellis obtained during recognition.

- The WG obtained after the search process is not a probabilistic and consistent graph.
- However, for some WG-based operations, it is necessary to normalize the WG.
- There are various graph normalization criteria.

First, we need to define the forward and backward functions:

$$\alpha_x(t, q) = \begin{cases} I(q) & \text{if } t = 0 \\ \sum_{q'} \alpha_x(t-1, q') \cdot P(q', x_t, q) & \text{otherwise} \end{cases}$$

$$\beta_x(t, q) = \begin{cases} F(q) & \text{if } t = |x| \\ \sum_{q'} \beta_x(t+1, q') \cdot P(q, x_{t+1}, q') & \text{otherwise} \end{cases}$$

Finally, the following statement can be proven:

$$P_A(x) = \sum_{q \in Q} \alpha_x(|x|, q) \cdot F(q) = \sum_{q \in Q} I(q) \cdot \beta_x(0, q)$$

The normalized weight for an arc (q', q) can be calculated as:

$$\psi(q', q) = \frac{P(q', q) \cdot \beta(q)}{\beta(q')}$$

➤ **Property 1:** Well-formed

$$\sum_{q \in Q} \psi(q', q) = 1 \quad \forall q' \in (Q - F);$$

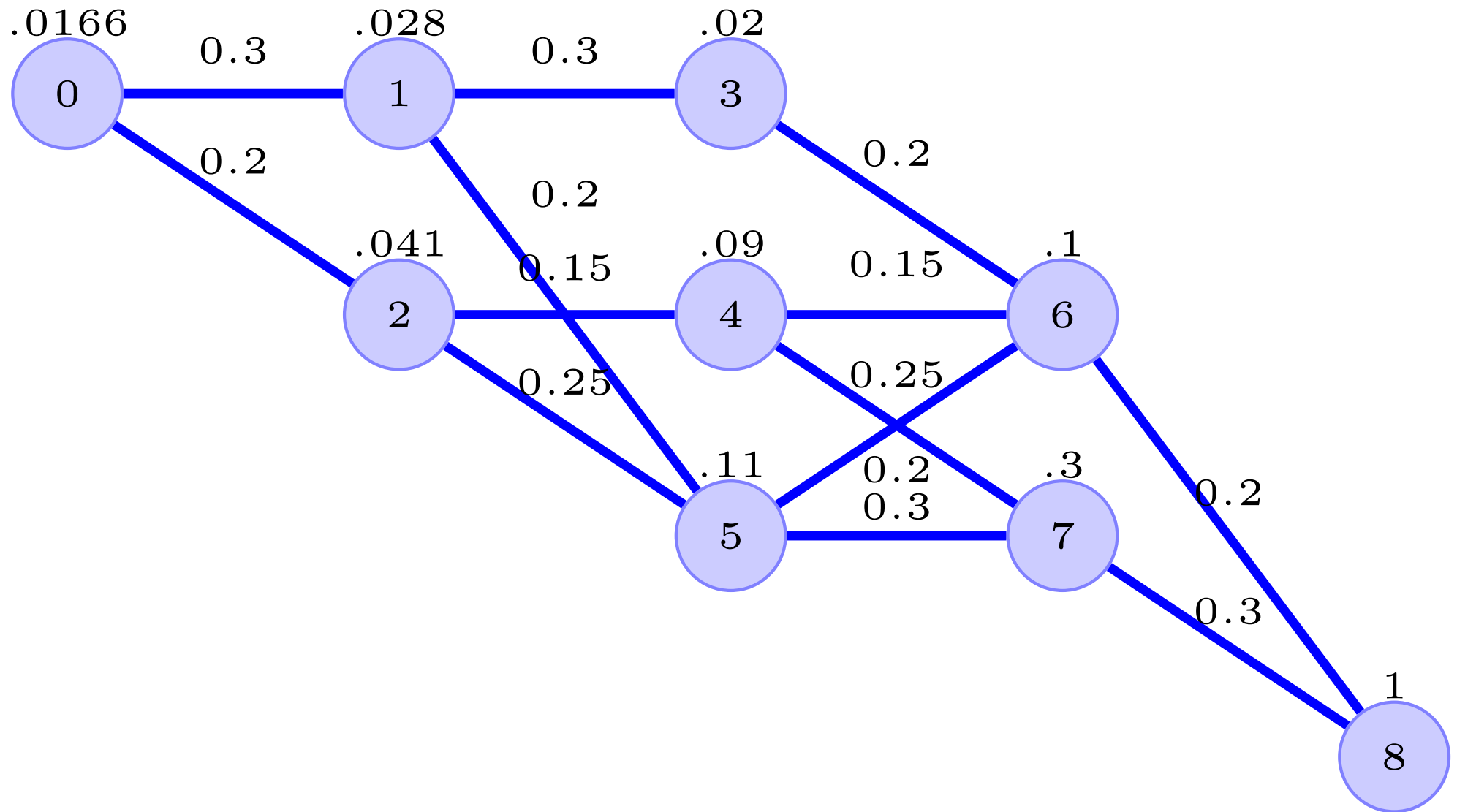
➤ **Property 2:** Consistency

$$\sum_{\phi \in \Phi} P(\phi) = 1,$$

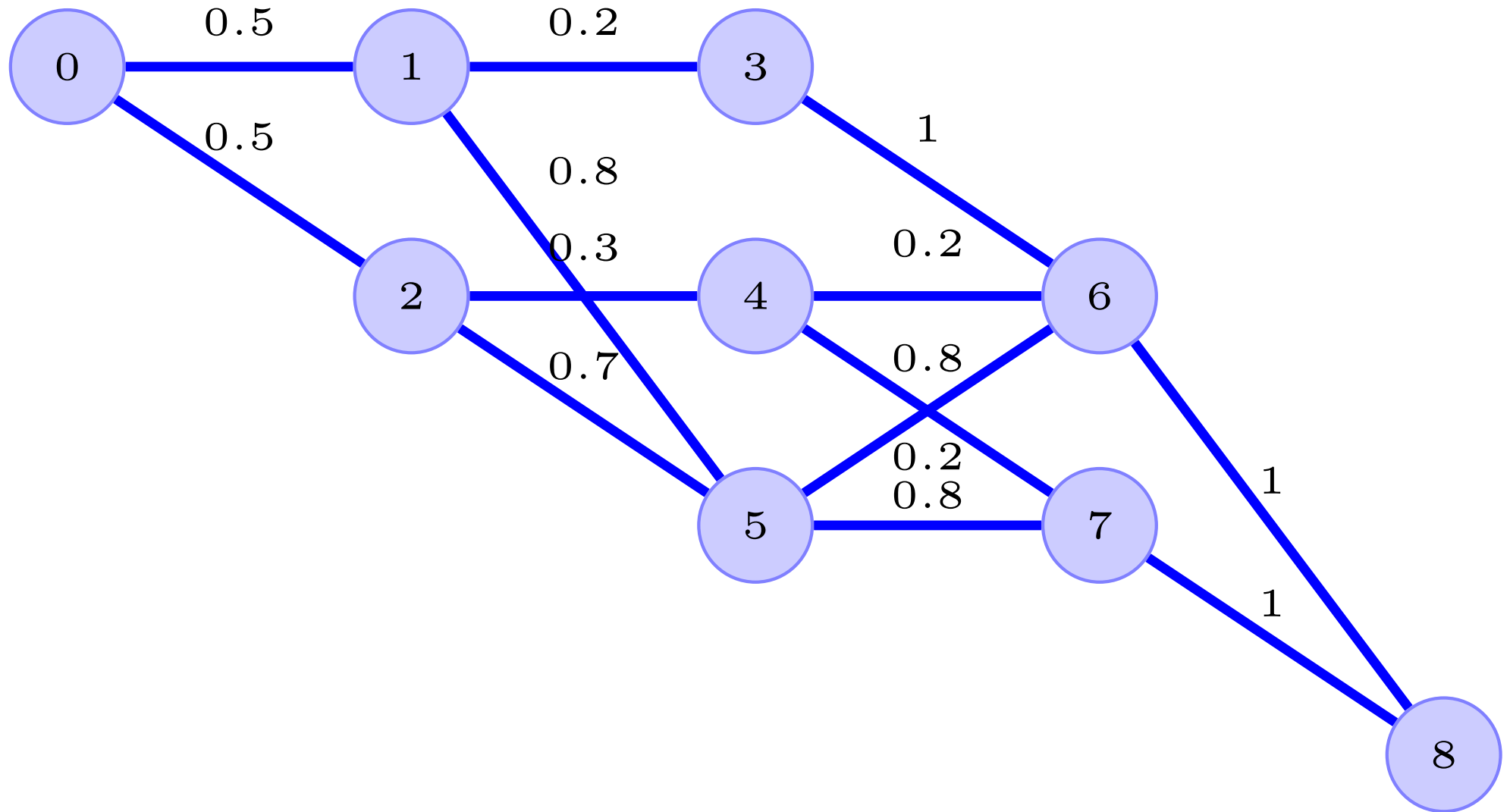
where Φ are all the paths of the WG.

➤ **Property 3:** The weight distribution of the sentences in the WG is not changed.

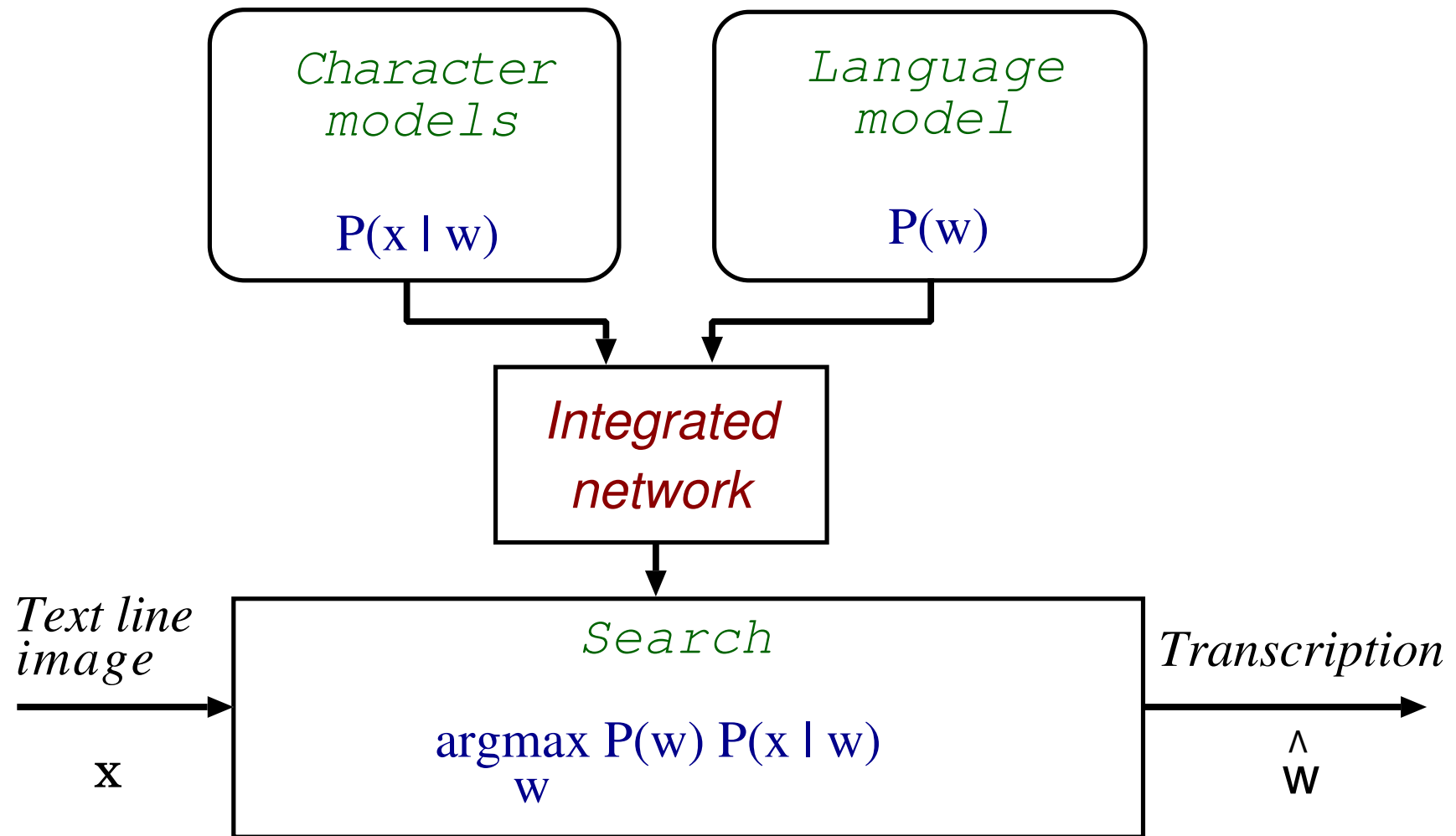
WORD GRAPHS: NORMALIZATION



WORD GRAPHS: NORMALIZATION



WORD GRAPHS: HTR INTEGRATION AND DECODIFICATION



Search engine:

THE VITERBI ALGORITHM (+ beam search + ...)

Handwritten Text Recognition (HTR)

Given an input handwritten text image, find its most likely written transcription.

- Units combine into higher level units: **morphological** → **lexical** → **syntactics**.
- Relationships between levels can be modeled by weighted graphs.
- Recognition: find the best path in a suitable product graph.
- Morphological, lexical and syntactical are modelled by homogeneous PFS models.
- All these PFS models can be easily integrated into a single global (huge) PFS model.
- This global PFS model accepts sequences of raw feature vectors and outputs strings of recognized words.

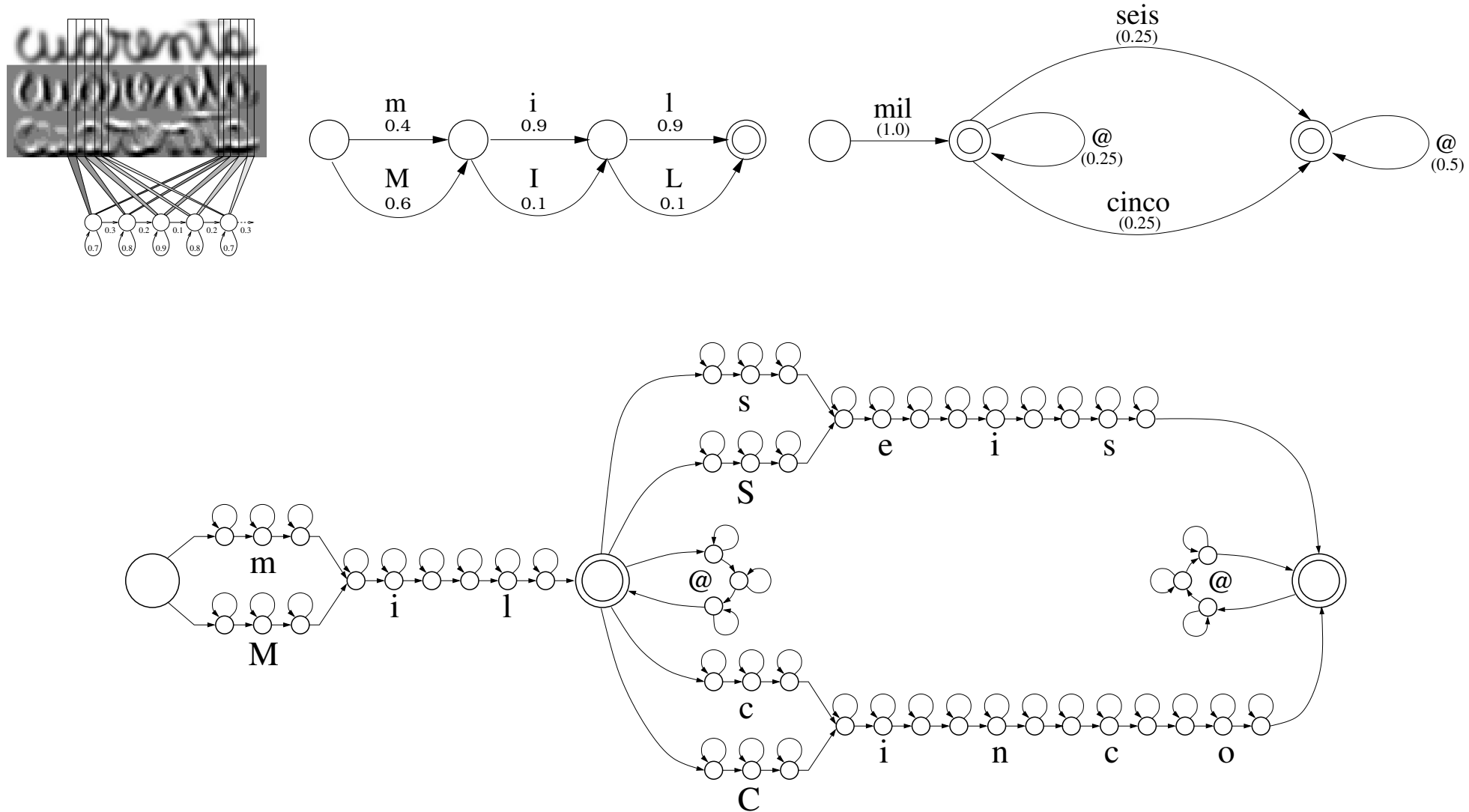
M Connectionist Temporal Classification (CTC) and hybrid NN/HMMs.

L Pronunciation dictionary transducer mapping character transcriptions to sequences.

S Language model weighted automaton.

$$\min (\det (M \circ L \circ G))$$

WORD GRAPHS: HTR INTEGRATION AND DECODIFICATION



Preliminaries

- The first step consists of calculating the probability of **the best path** from each state $q \in Q$ **to the final state** $f \in F$.

$$\Phi[q] = \max_{\pi \in P(q,f)} \{P(\pi) \cdot \rho(f)\}$$

- We consider pairs (q, p) of a state $q \in Q$ and a cumulative probability p .
- The algorithm uses a **priority queue** S containing the set of pairs (q, p) to examine next. The queue's ordering is based on Φ and defined by:

$$(q, p) > (q', p') \iff (p \cdot \Phi[q] > p' \cdot \Phi[q'])$$

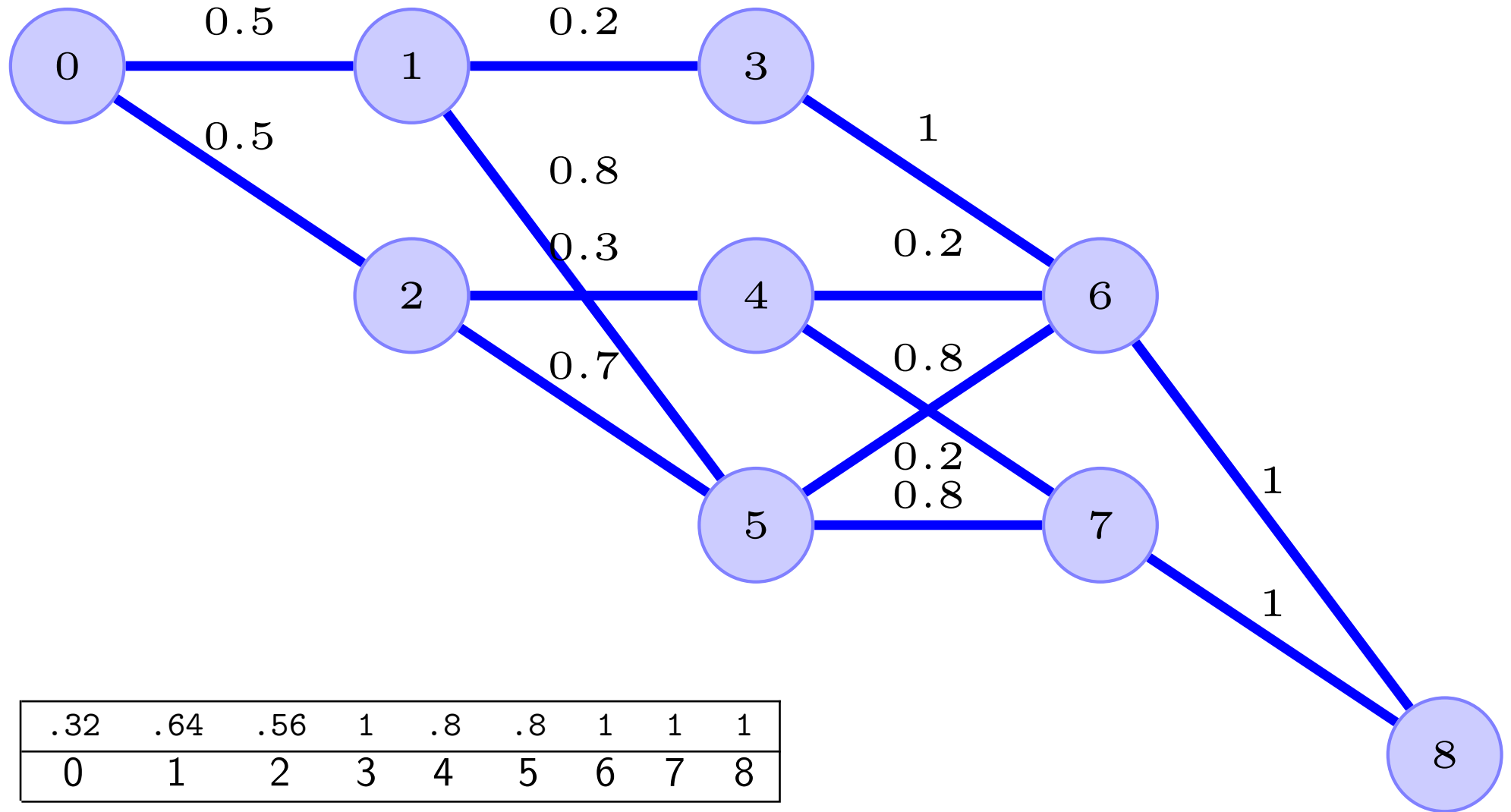
- For each state $q \in Q$, $r[q]$ gives the number of times a pair (q, p) with first state q has been extracted from S .
- $\pi[(q, p)]$ defines the predecessor in the path for pair (q, p) .
- $E[q]$ provides all the edges that start from state q .

Algorithm 2: N-best algorithm

```
for  $q : 0 \dots |Q| - 1$  do  $r[q] = 0$  ;  
 $\pi[(i, 1)] = \text{NIL}$  ;  
 $S = \{(i, 1)\}$  ;  
while  $S \neq \emptyset$  do  
     $(q, p) = \text{DEQUEUE}(S)$  ;  
     $r[q] = r[q] + 1$  ;  
    if (  $r[q] = N$  and  $q = f \in F$  ) then EXIT;  
    if (  $r[q] \leq N$  ) then  
        for  $e \in E(q)$  do  
             $p' = p \cdot w[e]$  ;  
             $\pi[(n(e), p')] = (q, p)$  ;  
            ENQUEUE (  $S, (n(e), p')$  ) ;
```

[M.Mohri and M.Riley: *An efficient algorithm for the N-best-strings problem*, 2002], for more details.

WORD GRAPHS BASED N-BEST SEARCH



WORD GRAPHS BASED N-BEST SEARCH

Φ	.32	.64	.56	1	.8	.8	1	1	1
	0	1	2	3	4	5	6	7	8
r	1	1	1	0	0	2	0	2	2

N=2

(0, 1)	1	1 · 0.5	$\pi(1, 0.5) \leftarrow (0, 1)$
	2	1 · 0.5	$\pi(2, 0.5) \leftarrow (0, 1)$
(1, 0.5)	3	0.5 · 0.2	$\pi(3, 0.1) \leftarrow (1, 0.5)$
	5	0.5 · 0.8	$\pi(5, 0.4) \leftarrow (1, 0.5)$
(5, 0.4)	6	0.4 · 0.2	$\pi(6, 0.08) \leftarrow (5, 0.4)$
	7	0.4 · 0.8	$\pi(7, 0.32) \leftarrow (5, 0.4)$
(7, 0.32)	8	0.32 · 1	$\pi(8, 0.32) \leftarrow (7, 0.32)$
(8, 0.32)			
(2, 0.5)	4	0.5 · 0.3	$\pi(4, 0.15) \leftarrow (2, 0.5)$
	5	0.5 · 0.7	$\pi(5, 0.35) \leftarrow (2, 0.5)$
(5, 0.35)	6	0.35 · 0.2	$\pi(6, 0.18) \leftarrow (5, 0.35)$
	7	0.35 · 0.8	$\pi(7, 0.28) \leftarrow (5, 0.35)$
(7, 0.28)	8	0.28 · 1	$\pi(8, 0.28) \leftarrow (7, 0.28)$
(8, 0.28)			

S

(0, 1)	(1, 0.5)	(2, 0.5)
	0.32	0.28
(3, 0.1)	(5, 0.4)	(6, 0.08)
0.1	0.32	0.08
(7, 0.32)	(8, 0.32)	(4, 0.15)
0.32	0.32	0.12
(5, 0.35)	(6, 0.07)	(7, 0.28)
0.28	0.07	0.28
(8, 0.28)		
0.28		

WORD GRAPHS BASED N-BEST SEARCH

